

Medical Appointment Booking System

Backend Software Design Document (SDD) Django · Django REST Framework · PostgreSQL · JWT

Backend Engineering Team

Version 1.0 — July 2026

Table of Contents

Document Control.....	6
Revision History.....	7
How to Use This Document.....	7
1. Executive Summary.....	8
1.1 Project Overview.....	8
1.2 Goals.....	8
1.3 Objectives.....	8
1.4 Scope.....	8
In Scope (Backend MVP).....	8
Out of Scope (this MVP; see Section 16 — Future Improvements).....	9
1.5 Expected Users.....	9
1.6 System Overview.....	9
2. Requirements Analysis.....	11
2.1 Functional Requirements (Restated for Engineering).....	11
2.2 Non-Functional Requirements.....	12
2.3 Business Rules.....	13
2.4 Hidden Requirements.....	13
2.5 Edge Cases.....	14
2.6 Risks.....	14

2.7 Assumptions.....	15
2.8 Missing Requirements (Flagged for Product Owner).....	16
2.9 Questions for Stakeholder Clarification.....	16
3. Backend Architecture.....	17
3.1 Overall Architecture.....	17
3.2 Layered Responsibilities.....	18
3.3 Django Apps.....	18
3.4 Recommended Project Folder Structure.....	18
3.5 App Dependency Graph.....	20
3.6 Recommended Packages.....	21
3.7 Coding Standards.....	21
3.8 Naming Conventions.....	21
3.9 Logging Strategy.....	22
3.10 Configuration Strategy.....	22
4. Database Design.....	24
4.1 Design Principles.....	24
4.2 Entity Relationship Diagram.....	25
4.3 Model Specifications.....	26
4.3.1 User (app: accounts).....	26
4.3.2 VerificationToken (app: accounts).....	27
4.3.3 Specialty (app: doctors).....	27
4.3.4 DoctorProfile (app: doctors).....	28
4.3.5 PatientProfile (app: patients).....	29
4.3.6 DoctorAvailability (app: doctors).....	30
4.3.7 Appointment (app: appointments).....	30
4.3.8 Notification (app: notifications).....	32
4.3.9 AuditLog (app: accounts).....	33
4.4 Relationship Summary.....	34
4.5 Database Optimization Recommendations.....	34
4.6 Deriving Bookable Slots (Not a Table, a Computation).....	34
4.7 Future Scalability Considerations.....	35
5. Authentication & Authorization.....	36

5.1 Registration Flow.....	36
5.2 Login Flow.....	36
5.3 JWT Lifecycle.....	37
5.4 Refresh Tokens.....	37
5.5 Email Verification.....	37
5.6 Password Reset.....	38
5.7 Permissions.....	38
5.8 Role-Based Access Summary.....	38
5.9 Object-Level Permissions.....	39
5.10 Security Considerations for This Section.....	39
6. REST API Design.....	40
6.1 Conventions.....	40
6.2 Standard Response Format.....	40
6.3 Standard Error Format.....	40
6.4 Standard Status Codes.....	41
6.5 Pagination.....	41
6.6 Filtering, Searching, Sorting.....	42
6.7 API Versioning.....	42
6.8 Endpoint Catalogue.....	43
6.8.1 Authentication (/api/v1/auth/).....	43
6.8.2 Accounts / Users (/api/v1/users/).....	43
6.8.3 Specialties (/api/v1/specialties/).....	44
6.8.4 Doctors (/api/v1/doctors/).....	45
6.8.5 Patients (/api/v1/patients/).....	46
6.8.6 Appointments (/api/v1/appointments/).....	46
6.8.7 Notifications & Audit (admin visibility).....	47
6.9 Detailed Request/Response Examples.....	48
6.9.1 POST /api/v1/auth/register/.....	48
6.9.2 POST /api/v1/auth/login/.....	48
6.9.3 GET /api/v1/doctors/{id}/slots/?date=2026-07-20.....	49
6.9.4 POST /api/v1/appointments/ (Booking — Patient).....	50
6.9.5 POST /api/v1/appointments/{id}/cancel/.....	50

6.9.6 POST /api/v1/appointments/{id}/reschedule/.....	51
6.9.7 POST /api/v1/appointments/{id}/reject/ (Doctor).....	51
7. Business Logic.....	53
7.1 Registration.....	53
7.2 Doctor Approval.....	54
7.3 Patient Management.....	55
7.4 Doctor Availability Management.....	55
7.5 Booking & Double-Booking Prevention.....	55
7.6 Appointment Lifecycle.....	57
7.7 Cancellation.....	57
7.8 Rescheduling.....	58
7.9 Notifications.....	58
7.10 Admin Operations.....	58
8. Validation Rules.....	59
8.1 User.....	59
8.2 DoctorProfile.....	59
8.3 PatientProfile.....	60
8.4 Specialty.....	60
8.5 DoctorAvailability.....	60
8.6 Appointment.....	60
8.7 VerificationToken.....	61
8.8 Field-Length & Format Quick Reference.....	61
9. Security.....	62
9.1 JWT Security.....	62
9.2 Permissions.....	62
9.3 Password Security.....	62
9.4 CORS.....	62
9.5 CSRF.....	62
9.6 Rate Limiting.....	62
9.7 Input Validation.....	63
9.8 SQL Injection Protection.....	63
9.9 XSS Protection.....	63

9.10 Secure File Uploads.....	63
9.11 Environment Variables & Secrets Management.....	63
9.12 OWASP Top 10 — Applied Mapping.....	64
10. Development Roadmap.....	65
10.1 Milestones.....	65
10.2 Dependency Graph.....	66
10.3 Recommended Implementation Order.....	66
10.4 Estimated Complexity Rationale.....	66
11. Team Distribution.....	67
11.1 Assignments.....	67
Developer 1 — Team Lead: Foundation & Identity.....	67
Developer 2 — Doctor Domain.....	67
Developer 3 — Patient Domain & Patient-Side Booking.....	67
Developer 4 — Doctor/Admin-Side Appointments & Notifications.....	68
11.2 Workload Summary.....	68
11.3 Merge Order (Critical Path).....	69
11.4 Code Review Checklist.....	69
12. Git Workflow.....	70
12.1 Branching Model.....	70
12.2 Branch Naming.....	70
12.3 Commit Conventions.....	70
12.4 Pull Request Workflow.....	71
12.5 Code Review Process.....	71
12.6 Release Strategy.....	71
13. Testing Strategy.....	72
13.1 Testing Pyramid for This Project.....	72
13.2 Unit Tests.....	72
13.3 Integration Tests.....	72
13.4 API Tests.....	72
13.5 Permission Tests.....	73
13.6 Authentication Tests.....	73
13.7 Appointment Tests.....	73

13.8 Factories & Fixtures.....	73
13.9 Coverage Goals.....	73
13.10 Testing Tools Summary.....	74
14. Deployment & DevOps Strategy.....	75
14.1 Environments.....	75
14.2 Containerization.....	75
14.3 CI/CD Pipeline.....	75
14.4 Database Migrations in Production.....	75
14.5 Configuration & Secrets in Production.....	75
14.6 Monitoring & Observability.....	76
14.7 Backups & Disaster Recovery.....	76
15. Developer Guide.....	77
15.1 Project Setup.....	77
15.2 Folder Explanation.....	77
15.3 Environment Variables.....	77
15.4 Coding Standards & Naming Conventions.....	78
15.5 Developer Onboarding Checklist (First Day).....	78
15.6 README Structure (Repository Root).....	78
15.7 Best Practices Recap.....	78
16. Future Improvements.....	79

Document Control

Field	Detail
Document Title	Medical Appointment Booking System — Backend Software Design Document
Version	1.0
Status	Draft for Development Kickoff
Audience	Backend development team, Technical Lead, QA
Related Documents	Product Requirements (React + Django), Frontend API Integration Guide
Backend Stack	Python 3.12, Django 5.x, Django REST Framework, PostgreSQL 16, SimpleJWT, Celery + Redis

Revision History

Version	Date	Author	Description
0.1	—	Software Architect	Initial draft from requirements analysis
1.0	July 2026	Software Architect	First complete release for development kickoff

How to Use This Document

This document is the single source of truth for backend implementation. It should be read in full by every backend developer before writing any code, and used as the reference for API contracts during frontend integration. Section 11 (Team Distribution) and Section 12 (Git Workflow) should be re-read at the start of every sprint. Any deviation from this design during implementation must be raised with the Technical Lead and reflected back into this document.

1. Executive Summary

1.1 Project Overview

The Medical Appointment Booking System is a full-stack web application that lets patients discover doctors, view real-time availability, and book, cancel, or reschedule appointments without a phone call. Doctors manage their own schedules and appointment queues, and administrators oversee the entire network of practitioners and patients. This document specifies the **backend** of that system: a Django + Django REST Framework API that the existing React frontend consumes.

The backend is the system of record and the enforcement point for every business rule described in the product requirements — who is allowed to do what, which slots are actually bookable, and how an appointment moves through its lifecycle. The frontend is treated purely as an API consumer; no frontend implementation detail is specified here beyond the contract the API exposes.

1.2 Goals

- Provide a secure, role-aware REST API that supports three distinct user experiences (Patient, Doctor, Admin) from a single, consistent data model.
- Guarantee data integrity for the single highest-risk operation in the system: **preventing two patients from booking the same doctor slot.**
- Ship an MVP that a 4-person backend team can build in parallel with minimal merge conflicts and a predictable integration order.
- Produce an API contract detailed enough that frontend integration requires no backend guesswork.

1.3 Objectives

1. Implement JWT-based authentication with mandatory email verification before login.
2. Implement role-based access control (RBAC) for Admin, Doctor, and Patient roles, enforced at both the view/permission layer and the object level.
3. Implement a doctor availability model that supports recurring weekly schedules and derives real-time bookable slots.
4. Implement an appointment lifecycle (pending → confirmed/rejected → completed/cancelled/rescheduled) with full audit visibility for admins.
5. Implement transactional, race-condition-safe booking that cannot double-book a doctor.
6. Send email notifications for the events the requirements call out explicitly (registration confirmation, booking confirmation), with a notification model general enough to extend to reminders and cancellations.
7. Expose Django Admin as the administrative control surface for the MVP, backed by the same permission and audit rules as the API.

1.4 Scope

In Scope (Backend MVP)

- Custom user model with three roles, email verification, JWT auth (access + refresh), password reset.

- Admin approval/blocking workflow for doctor (and patient) accounts.
- Specialty, Doctor Profile, and Patient Profile management.
- Doctor weekly availability configuration.
- Doctor discovery with filtering (specialty, name) and pagination.
- Appointment booking, cancellation, and rescheduling with double-booking prevention.
- Doctor-side appointment approval/rejection and notes.
- Email notifications for registration confirmation and booking confirmation.
- Django Admin configuration for all models with role-appropriate actions.
- Automated test suite (unit + API + permission tests).

Out of Scope (this MVP; see Section 16 — Future Improvements)

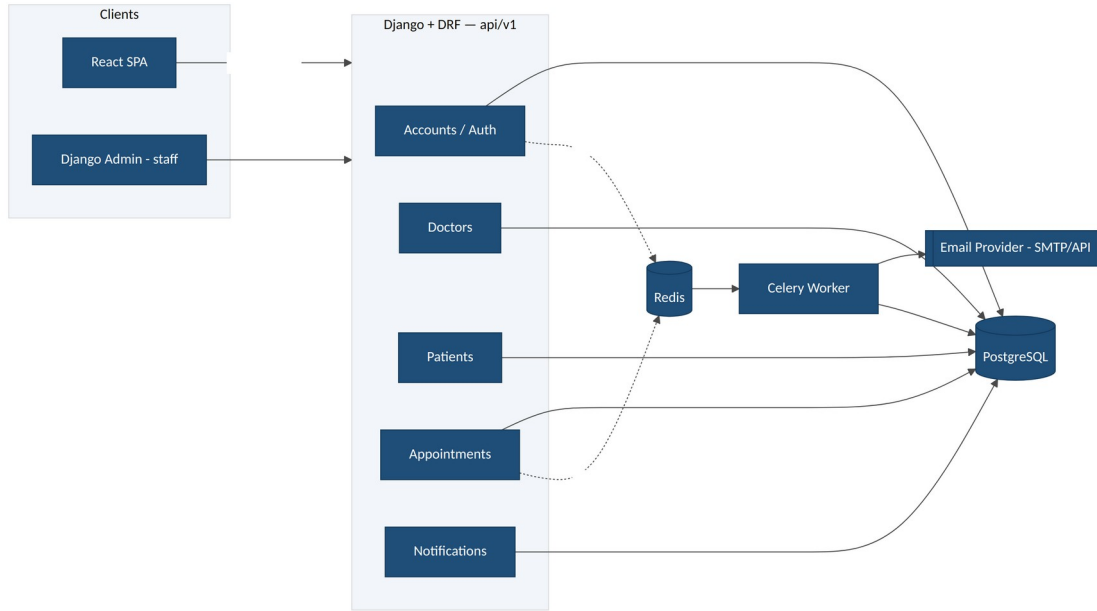
- Payments and billing.
- Video consultation / telemedicine.
- SMS or push notifications (email only, per requirements).
- Native mobile apps.
- Multi-clinic / multi-tenant support.
- Real-time chat between patient and doctor.
- Frontend implementation details (React is treated as an API consumer only).

1.5 Expected Users

Role	Who they are	Primary need
Patient	General public creating an account to seek care	Find a suitable doctor and book a slot with confidence it will hold
Doctor	Licensed practitioners onboarded by an admin	Control their schedule and manage an incoming queue of requests
Admin	Internal staff operating the platform	Keep the doctor directory trustworthy (approvals) and have full visibility into activity

1.6 System Overview

At a high level, the backend is a layered Django project exposing a single versioned REST API (`/api/v1/`) secured by JWT bearer tokens. Six Django apps own distinct slices of the domain (accounts, doctors, patients, appointments, notifications, core), coordinated through Django's ORM against a single PostgreSQL database. Asynchronous work — primarily transactional email — is offloaded to Celery workers backed by Redis so that request/response cycles stay fast and booking transactions are never blocked on SMTP latency.



The remainder of this document works from these principles: Section 2 pressure-tests the requirements, Sections 3–9 specify the system itself, Sections 10–14 specify how the team builds and ships it, and Sections 15–16 hand the finished system to future maintainers.

2. Requirements Analysis

This section restates the product requirements in engineering terms, then does the work the original requirements document does not: it surfaces the rules that are implied but never stated, the edge cases that will break a naive implementation, and the questions that need an answer before — or very early in — Sprint 1.

2.1 Functional Requirements (Restated for Engineering)

#	Requirement	Source
FR-1	Patients and doctors can self-register; admins are seeded/created directly	Product requirements §1
FR-2	Registration triggers a mandatory email confirmation before the account can log in	Product requirements §1
FR-3	Three roles exist: Admin, Doctor, Patient, each with a distinct permission set	Product requirements §1
FR-4	Authentication is JWT-based via SimpleJWT	Product requirements §1
FR-5	Admin can view all users, approve/block doctors and patients	Product requirements §2
FR-6	Admin has CRUD over specialties, doctors, and system configuration	Product requirements §2
FR-7	Admin can view all appointments system-wide	Product requirements §2
FR-8	Doctor can define a weekly availability schedule	Product requirements §3
FR-9	Doctor can view upcoming and past appointments	Product requirements §3
FR-10	Doctor can manage their own profile (specialty, bio, contact)	Product requirements §3
FR-11	Doctor can approve/reject an appointment and attach notes	Product requirements §3
FR-12	Patient can browse/filter doctors by specialty and name	Product requirements §4
FR-13	Patient can view a specific doctor's availability	Product requirements §4
FR-14	Patient can book a date/time slot	Product requirements §4

#	Requirement	Source
FR-15	Patient can view, cancel, or reschedule their own appointments	Product requirements §4
FR-16	Patient can edit their own profile	Product requirements §4
FR-17	Availability is validated in real time; double-booking is impossible	Product requirements §5
FR-18	Appointments have a status: pending, confirmed, completed, cancelled	Product requirements §5
FR-19	Email confirmation is sent for bookings	Product requirements §6

2.2 Non-Functional Requirements

Category	Requirement
Security	All write endpoints require authentication; sensitive endpoints require ownership or admin role; passwords hashed with Django's PBKDF2/Argon2; JWTs short-lived with rotating refresh tokens
Data integrity	Double-booking must be impossible even under concurrent requests — enforced at the database layer, not only in application code
Performance	Doctor search and availability endpoints must respond in well under 300ms at MVP scale (single-digit thousands of doctors, tens of thousands of appointments)
Availability	Email delivery must never block or fail an API request — sent asynchronously with retry
Auditability	Every admin action that changes another user's state (approve, block) must be recorded
Maintainability	Business logic lives in a service layer, not in views or signals, so it is unit-testable in isolation
Portability	Environment-driven configuration (12-factor); no secrets in source control
API consistency	One versioned API (<code>/api/v1/</code>), one response envelope, one error format, applied uniformly
Internationalization	Out of scope for MVP, but field lengths and encoding must not preclude adding it later (UTF-8 throughout, no hard-coded locale formatting in stored data)

2.3 Business Rules

- A doctor account cannot accept bookings until an admin approves it (`account_status = ACTIVE`). Patients never see unapproved doctors in search results.
- A blocked account (doctor or patient) cannot authenticate, regardless of prior email verification.
- An appointment slot is only bookable if it falls within the doctor's active `DoctorAvailability` window **and** does not overlap an existing non-cancelled appointment for that doctor.
- Only the owning patient can cancel/reschedule their own appointment; only the owning doctor (or an admin) can approve/reject/complete it.
- A cancelled or rejected appointment frees its slot immediately.
- Rescheduling is modeled as cancel-old + create-new, linked by a `previous_appointment` reference, so the audit trail of the original booking is never lost.
- Doctors cannot be deleted once they have appointment history; they are deactivated (soft state), never hard-deleted, to preserve patient appointment history integrity.
- A patient cannot book a slot in the past, and (business rule, confirm with stakeholders — see 2.9) cannot book less than a configurable minimum lead time (e.g., 1 hour) before the slot start.

2.4 Hidden Requirements

These are requirements the product document implies but never states outright. They must be designed for even though no one asked for them explicitly:

- **Email verification token expiry and resend.** “Sending a confirmation email is a must” implies a token, an expiry, and — inevitably — a “resend confirmation” endpoint, because some emails will land in spam or expire before the user clicks.
- **Password reset**, referenced nowhere in the functional requirements but required by any real authentication system and explicitly named in the SDD prompt's own Section 5 — treated here as an implicit requirement of “Authentication” (FR-4).
- **Doctor rejection reason.** If a doctor can reject an appointment, the patient needs to know *why* — otherwise support tickets pile up. `doctor_notes` / `cancellation_reason` fields exist for this.
- **What happens to a doctor's future appointments when an admin blocks them?** Not specified. The design assumes existing confirmed/pending appointments are **not** auto-cancelled (a human decision is safer) but the doctor cannot be booked further, and admins are surfaced a warning listing affected appointments (see 2.6, risk R-3).
- **Timezone.** The requirements never mention timezone, but a doctor and patient could plausibly be in different zones even within one country. MVP assumption: **single-timezone deployment** (confirm with stakeholders, 2.9), all times stored as `TIMESTAMPZ`/naive-local consistently in UTC internally, displayed in one configured local timezone by the frontend.
- **Concurrent booking of the same slot.** “Real-time validation of availability” is stated, but the actual hard requirement — a unique database constraint, not just an application check — is implicit and is treated as first-class in Section 4 and Section 7.
- **Rate limiting on auth endpoints** to prevent credential stuffing / registration spam — implied by any production-grade auth system, not stated.

- **Soft delete / audit trail for admin CRUD** on specialties and doctors, so historical appointments still display a specialty name even if it is later renamed or deactivated.

2.5 Edge Cases

Edge Case	Handling
Patient books the last open slot of a doctor who is simultaneously being blocked by an admin	Booking and blocking both run in DB transactions; whichever commits first wins. The design does not need to prevent this — it needs the <i>result</i> to be consistent, which the transaction isolation already guarantees.
Doctor changes their weekly availability <i>after</i> patients have already booked slots that fall outside the new schedule	Existing appointments are not retroactively invalidated. Availability changes only affect future slot generation. This must be documented for the doctor UI.
Patient tries to reschedule a PENDING appointment	Allowed — reschedule cancels the pending appointment and creates a new pending one with the doctor's clock reset to pending.
Patient tries to reschedule a COMPLETED or already-CANCELLED appointment	Rejected with 409 Conflict — only PENDING/CONFIRMED appointments are reschedulable.
Two overlapping availability windows defined by the same doctor for the same day	Rejected at the serializer level (validation, not just DB constraint) with a clear error before it ever reaches the database.
A doctor's <code>slot_duration_minutes</code> does not evenly divide their availability window	Rejected at validation time with an explicit error rather than silently truncating the last slot.
Patient account is blocked while they have pending appointments	Pending/confirmed appointments remain visible to the doctor/admin; the patient simply can no longer authenticate to interact with them.
Email delivery fails (SMTP down)	The booking/registration itself still succeeds; the Notification row is marked FAILED and retried by Celery with backoff. The user is never blocked by a downstream email provider outage.
Doctor tries to book — role should not permit doctors to book appointments as patients	Enforced by permission classes; a doctor account cannot hit the patient booking endpoint even if UUIDs are guessed.

2.6 Risks

ID	Risk	Impact	Mitigation
R-1	Double-booking under concurrent requests	High — core trust of the product	DB-level unique constraint + <code>select_for_update</code> transactional booking

ID	Risk	Impact	Mitigation
			(Section 4, Section 7)
R-2	Email provider outage delays or drops confirmations	Medium	Async delivery via Celery with retry/backoff; Notification audit table to detect failures
R-3	Admin blocks a doctor with an active appointment backlog, patients left confused	Medium	Admin UI/endpoint surfaces affected appointment count before confirming a block (see 2.4)
R-4	JWT refresh token theft/replay	Medium-High	Short-lived access tokens, rotating + blacklisted refresh tokens, HTTPS-only, Secure/HttpOnly storage guidance for frontend
R-5	Unbounded doctor search results at scale	Low at MVP, grows over time	Mandatory pagination + DB indexes from day one (Section 4)
R-6	Team of 4 stepping on each other in shared files (<code>urls.py</code> , <code>settings.py</code>)	Medium — velocity risk, not a system risk	Explicit module ownership and merge order (Section 11)
R-7	Scope creep of “notifications” into a full messaging system mid-sprint	Medium	Notification model is deliberately generic now; SMS/push explicitly deferred to Section 16

2.7 Assumptions

1. Single country/timezone deployment for MVP (confirm — 2.9).
2. Admin accounts are created via Django management command / `createsuperuser`, not public self-registration.
3. One doctor has exactly one specialty for MVP (a doctor practicing two specialties is a future improvement).
4. Appointment slot granularity is fixed per doctor (e.g., all 30-minute slots for that doctor), not variable per booking.
5. “Real-time” availability means computed on request, not push/WebSocket-based.
6. Email is the only notification channel for MVP, sent via a transactional email provider (e.g., SES, SendGrid, Mailgun) through Django’s email backend.

7. The React frontend is responsible for timezone display formatting; the backend is the source of truth in a single canonical timezone.
8. Currency/consultation fee, if collected, is informational only for MVP (no payment processing).

2.8 Missing Requirements (Flagged for Product Owner)

- No requirement defines appointment **duration** business rules (fixed per doctor? per specialty? patient-selectable?). Assumption 4 above is used until clarified.
- No requirement defines a **maximum booking horizon** (can a patient book 2 years in advance?). A sane default (e.g., 60–90 days) is recommended, pending confirmation.
- No requirement defines **doctor no-show** / **patient no-show** handling — a fourth outcome distinct from “cancelled.”
- No requirement defines whether patients can see the identity of past cancelling/rescheduling actor (themselves vs. clinic-initiated).
- No mention of **search by location** — only specialty and name are named; geographic search is explicitly out of scope for MVP.

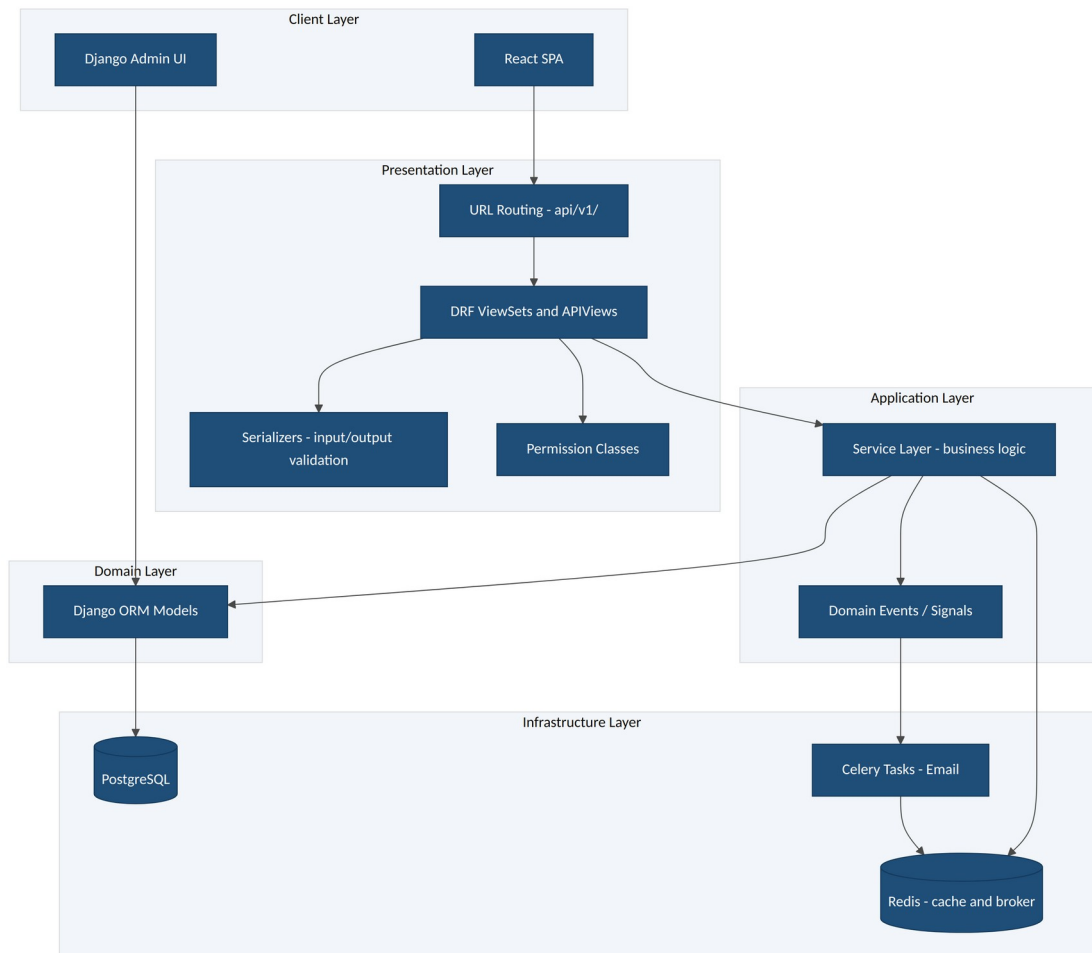
2.9 Questions for Stakeholder Clarification

1. Is the platform single-timezone for MVP, and if not, how should timezones be captured for doctors vs. patients?
2. What is the minimum lead time before a slot becomes unbookable (e.g., must be booked at least 1 hour in advance)?
3. What is the maximum booking horizon (how far in the future can a patient book)?
4. Should a blocked doctor’s existing pending/confirmed appointments be auto-cancelled, or left for manual admin handling?
5. Can a doctor hold more than one specialty?
6. Is appointment duration fixed per doctor, or should patients be able to request a longer/shorter slot?
7. Should patients be able to see which doctors have been “blocked” historically, e.g., for transparency on a past appointment with a now-blocked doctor?
8. Is a “no-show” status needed for MVP, or is it acceptable to fold it into “cancelled” for now?

3. Backend Architecture

3.1 Overall Architecture

The backend is a modular monolith: one Django project, multiple Django apps, one PostgreSQL database. A modular monolith is deliberately chosen over microservices — at MVP scale, microservices would add operational overhead (service discovery, distributed transactions for the exact booking consistency problem this project cares most about) without a corresponding benefit. App boundaries are drawn so that a future extraction into services remains possible if the product ever needs it.



Why a service layer. DRF encourages putting logic directly in `ViewSet` methods or `serializer.create()`. For this project that is not sufficient: booking, cancellation, doctor approval, and rescheduling are multi-step operations with transactional and validation requirements that need to be unit-tested without spinning up HTTP requests, and need to be reusable from Django Admin actions and management commands, not only from the API. Each app therefore has a `services.py` (or `services/` package) containing plain Python functions/classes that views call into. Views stay thin: parse input via serializer, call a service function, serialize the result.

3.2 Layered Responsibilities

Layer	Responsibility	Lives in
Presentation	HTTP contract: routing, request/response shape, permission gate	<code>views.py</code> , <code>serializers.py</code> , <code>urls.py</code> , <code>permissions.py</code>
Application	Business rules, orchestration, transactions	<code>services.py</code>
Domain	Data shape, model-level validation (<code>clean()</code>), relationships	<code>models.py</code>
Infrastructure	Persistence, caching, async task execution, email	<code>tasks.py</code> , Django settings, Celery config

3.3 Django Apps

App	Owns	Depends on
<code>core</code>	Base models (<code>TimeStampedModel</code>), shared permissions, pagination classes, standard response/error formatting, custom exception handler	— (no internal dependencies)
<code>accounts</code>	Custom <code>User</code> model, registration, email verification, JWT auth views, password reset, admin user management, <code>AuditLog</code>	<code>core</code>
<code>doctors</code>	<code>Specialty</code> , <code>DoctorProfile</code> , <code>DoctorAvailability</code> , doctor search	<code>core</code> , <code>accounts</code>
<code>patients</code>	<code>PatientProfile</code>	<code>core</code> , <code>accounts</code>
<code>appointments</code>	<code>Appointment</code> , booking/cancel/reschedule/approve services, slot computation	<code>core</code> , <code>accounts</code> , <code>doctors</code> , <code>patients</code>
<code>notifications</code>	<code>Notification</code> , email templates, Celery send tasks	<code>core</code> , <code>accounts</code>

3.4 Recommended Project Folder Structure

```
medbook_backend/
├── manage.py
├── requirements/
```

```

|   |— base.txt
|   |— dev.txt
|   |— prod.txt
|— config/                                # Django project package
|   |— __init__.py
|   |— settings/
|       |— base.py
|       |— dev.py
|       |— prod.py
|       |— test.py
|   |— urls.py                            # includes each app's urls under
/api/v1/
|   |— celery.py
|   |— asgi.py
|   |— wsgi.py
|— core/
|   |— models.py                        # TimeStampedModel, SoftDeleteModel
|   |— permissions.py                  # IsAdmin, IsDoctor, IsPatient,
IsOwnerOrAdmin
|   |— pagination.py
|   |— exceptions.py                  # custom exception handler
|   |— response.py                    # standard envelope helpers
|   |— tests/
|— accounts/
|   |— models.py                      # User, VerificationToken, AuditLog
|   |— serializers.py
|   |— services.py                    # register(), verify_email(),
reset_password()
|   |— views.py
|   |— urls.py
|   |— permissions.py
|   |— admin.py
|   |— managers.py                    # CustomUserManager
|   |— tests/
|— doctors/
|   |— models.py                      # Specialty, DoctorProfile,
DoctorAvailability
|   |— serializers.py
|   |— services.py                    # compute_available_slots()
|   |— views.py
|   |— urls.py
|   |— admin.py
|   |— tests/
|— patients/
|   |— models.py                      # PatientProfile

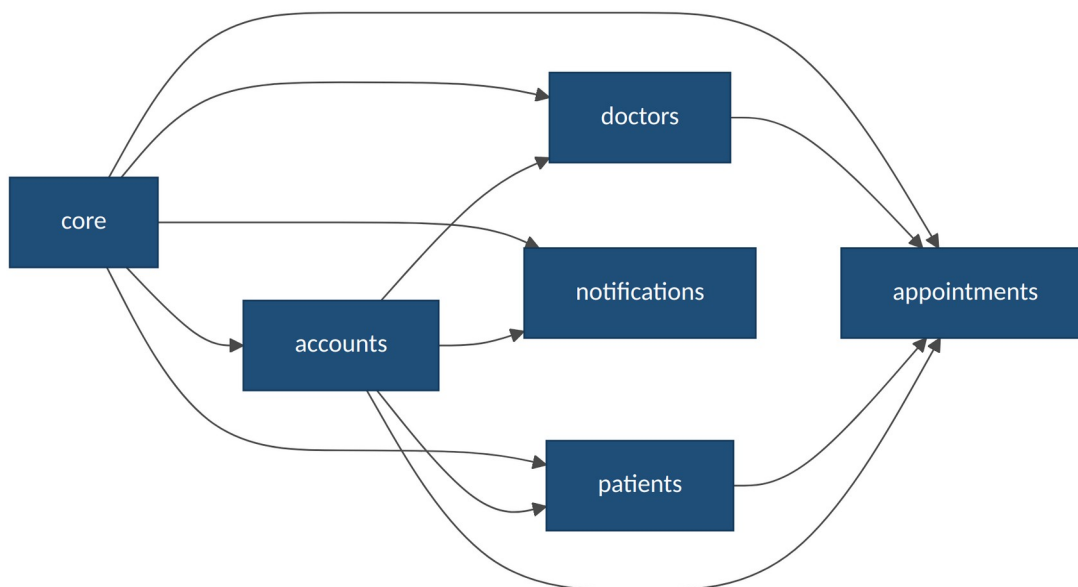
```

```

├── serializers.py
├── views.py
├── urls.py
├── admin.py
├── tests/
├── appointments/
│   ├── models.py           # Appointment
│   ├── serializers.py
│   └── services.py         # book(), cancel(), reschedule(),
│                           approve(), reject(), complete()
├── views.py
├── urls.py
├── permissions.py
├── admin.py
├── tests/
├── notifications/
│   ├── models.py           # Notification
│   ├── tasks.py            # Celery tasks
│   ├── services.py         # notify_booking_confirmed(), etc.
│   ├── templates/emails/
│   ├── admin.py
│   └── tests/
├── docs/
└── openapi/

```

3.5 App Dependency Graph



No cycles exist. `appointments` is intentionally the most-depended-upon app (it is the integration point) and is therefore the app most likely to cause merge conflicts — see Section 11 for how ownership is split to minimize that.

3.6 Recommended Packages

Package	Purpose
<code>django-rest-framework</code>	REST API framework
<code>django-rest-framework-simplejwt</code>	JWT authentication, refresh/rotation, blacklist
<code>django-filter</code>	Declarative filtering for doctor search
<code>psycopg[binary]</code>	PostgreSQL adapter
<code>celery+redis</code>	Async task queue for email sending
<code>django-cors-headers</code>	CORS for the React frontend origin
<code>drf-spectacular</code>	OpenAPI 3 schema generation from DRF views (keeps Section 6 contract enforceable in CI)
<code>python-decouple</code> or <code>django-environ</code>	Environment-variable configuration
<code>Pillow</code>	Image handling for profile pictures
<code>factory_boy+faker</code>	Test data factories (Section 13)
<code>pytest-django</code>	Test runner
<code>sentry-sdk</code>	Error monitoring (recommended pre-production)

3.7 Coding Standards

- **Formatting:** `black` (line length 100) + `isort`, enforced by a pre-commit hook and CI check — not a matter of taste, not up for per-PR debate.
- **Linting:** `flake8` or `ruff`; CI fails the build on lint errors.
- **Type hints:** required on all `services.py` function signatures; encouraged elsewhere. `mypy` is optional for MVP but recommended once the team is comfortable.
- **Docstrings:** every service function has a docstring stating pre-conditions and raised exceptions (what the view layer must translate into HTTP errors).
- **No business logic in views or serializers' `.save()`** beyond simple field assembly — call into `services.py`.
- **No raw SQL** unless a service function has a documented performance justification and a code comment linking to it.
- **Fat models, thin views, explicit services** — the project's one-line architecture rule.

3.8 Naming Conventions

Element	Convention	Example
Model	PascalCase, singular	<code>DoctorAvailability</code>
Field	snake_case	<code>appointment_date</code>
Foreign key field	snake_case singular,	<code>doctor, patient</code>

Element	Convention	Example
URL path	referencing the related model kebab-case, plural resource nouns	/api/v1/doctor-availabilities/
Serializer	<Model>Serializer, <Model>CreateSerializer, <Model>ListSerializer when shapes diverge	AppointmentCreateSerializer
Service function	verb-first, snake_case	book_appointment(), approve_appointment()
Permission class	Is<Role>, Is<Role>Or<Role>, <Verb>Permission	IsOwnerPatientOrAdmin
Celery task	<domain>_<action>_task	notifications_send_email_task
Test file	test_<unit>.py	test_booking_service.py
Git branch	see Section 12	feature/appointments-booking-service

3.9 Logging Strategy

- Structured JSON logging in production (python-json-logger or Django's built-in with a JSON formatter) so logs are queryable in the aggregator (CloudWatch/ELK/whatever the deployment target provides).
- Every request gets a `request_id` (middleware-generated UUID) propagated into all log lines and returned in the `X-Request-ID` response header, so a frontend bug report can be matched to backend logs.
- Log levels: **INFO** for business events (appointment booked/cancelled, doctor approved), **WARNING** for recoverable failures (email retry), **ERROR** for unhandled exceptions (also sent to Sentry), **DEBUG** only in local development.
- Never log passwords, tokens, or full JWTs. Log user IDs, not PII, wherever a business event needs an actor reference.
- The service layer, not the view layer, is responsible for emitting business-event logs — this keeps the log statement next to the business rule it documents.

3.10 Configuration Strategy

- Twelve-factor: all environment-specific values (`SECRET_KEY`, `DATABASE_URL`, `EMAIL_*`, `JWT_SIGNING_KEY`, `REDIS_URL`, `CORS_ALLOWED_ORIGINS`) come from environment variables, never hard-coded, never committed.
- `config/settings/base.py` holds everything environment-agnostic; `dev.py`, `prod.py`, `test.py` import from it and override only what differs.

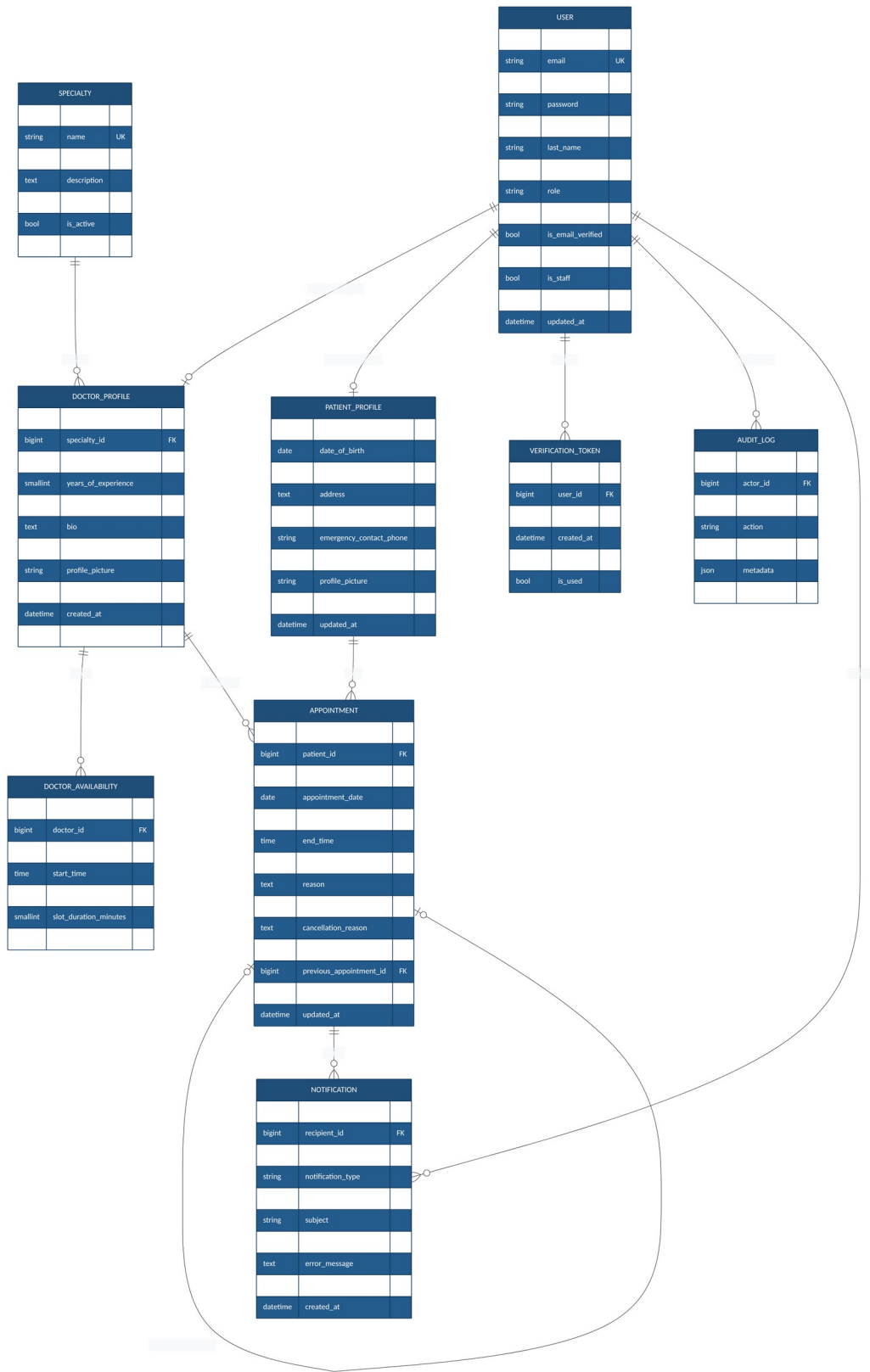
- `.env.example` checked into the repo documents every required variable with a safe placeholder value; `.env` itself is git-ignored.
- Secrets in production are injected via the hosting platform's secret manager (see Section 14), never baked into the Docker image.

4. Database Design

4.1 Design Principles

- One custom `User` model (`AbstractUser` subclass) is the root of identity; `DoctorProfile` and `PatientProfile` extend it via one-to-one relationships rather than multi-table inheritance, keeping auth concerns (`accounts` app) separate from domain profile data.
- Every table includes `created_at/updated_at` via a shared `TimeStampedModel` abstract base in `core`.
- Soft-delete (`is_active / deleted_at`) is used for records with historical dependents (`Specialty`, `DoctorProfile`); hard delete is acceptable for records with no dependents (`VerificationToken`, `Notification` past retention window).
- The single most important database rule in this entire system is the partial unique constraint on `Appointment` that makes double-booking structurally impossible (4.3.7). Every other constraint is important; this one is load-bearing.

4.2 Entity Relationship Diagram



4.3 Model Specifications

4.3.1 User (app: accounts)

Purpose. Single identity table for all three roles; extends `AbstractUser` and swaps the login field to email. `is_staff/is_superuser` are reserved for Django Admin access (Admin role).

Field	Type	Notes
<code>id</code>	<code>BigAutoField</code>	PK
<code>email</code>	<code>EmailField(unique=True)</code>	USERNAME_FIELD
<code>username</code>	<code>CharField(150, unique=True)</code>	kept for Django internals/admin display; auto-derived from email on registration
<code>first_name, last_name</code>	<code>CharField(150)</code>	required
<code>phone_number</code>	<code>CharField(20)</code>	validated via <code>RegexValidator</code> (E.164-ish)
<code>role</code>	<code>CharField(10, choices=Role)</code>	ADMIN / DOCTOR / PATIENT; immutable after creation (enforced in serializer, not just convention)
<code>account_status</code>	<code>CharField(20, choices=AccountStatus)</code>	PENDING_APPROVAL / ACTIVE / BLOCKED
<code>is_email_verified</code>	<code>BooleanField(default=False)</code>	gates login
<code>is_active</code>	<code>BooleanField(default=True)</code>	Django-standard login gate, kept in sync with <code>account_status != BLOCKED</code>
<code>is_staff, is_superuser</code>	<code>BooleanField</code>	Django built-in, True only for Admin
<code>date_joined, last_login</code>	<code>DateTimeField</code>	Django built-in
<code>updated_at</code>	<code>DateTimeField(auto_now=True)</code>	from <code>TimeStampedModel</code>

Relationships. 1-1 to `DoctorProfile/PatientProfile` (whichever matches `role`); 1-many to `Appointment` (as patient or doctor), `Notification`, `VerificationToken`, `AuditLog`.

Indexes. `email` (unique, implicit), (`role`, `account_status`) composite — this is the query Admin’s “pending doctor approvals” screen runs constantly.

Constraints. `CheckConstraint` that `is_staff=True` only when `role='ADMIN'`.

Validators. `phone_number` regex; `role` immutability enforced at the serializer/service layer on update.

4.3.2 VerificationToken (app: accounts)

Purpose. Backs both email verification and password reset with one reusable model instead of two near-identical ones.

Field	Type	Notes
<code>token</code>	<code>UUIDField(primary_key=True, default=uuid4)</code>	unguessable, used in the emailed link
<code>user</code>	<code>ForeignKey(User, on_delete=CASCADE)</code>	
<code>purpose</code>	<code>CharField(20, choices=['EMAIL_VERIFICATION', 'PASSWORD_RESET'])</code>	
<code>created_at</code>	<code>DateTimeField(auto_now_add=True)</code>	
<code>expires_at</code>	<code>DateTimeField</code>	set on creation: +24h for verification, +1h for reset
<code>is_used</code>	<code>BooleanField(default=False)</code>	one-time use

Relationships. Many-to-one to `User`.

Indexes. (`user`, `purpose`, `is_used`) — used to invalidate/replace outstanding tokens when a new one is requested.

Constraints. None beyond FK; expiry/one-time-use enforced at the service layer (`verify_email()`, `reset_password()` reject expired or used tokens with a specific error).

4.3.3 Specialty (app: doctors)

Purpose. Admin-managed taxonomy patients filter doctors by.

Field	Type	Notes
<code>id</code>	<code>BigAutoField</code>	PK
<code>name</code>	<code>CharField(100, unique=True)</code>	e.g., “Cardiology”
<code>slug</code>	<code>SlugField(unique=True)</code>	auto-generated from name, used in URLs/filters
<code>description</code>	<code>TextField(blank=True)</code>	
<code>icon</code>	<code>CharField(50, blank=True)</code>	icon identifier for the frontend

Field	Type	Notes
<code>is_active</code>	<code>BooleanField(default=True)</code>	soft delete — deactivating hides it from new-doctor onboarding without breaking historical <code>DoctorProfile.specialty</code> references
<code>created_at</code>	<code>DateTimeField(auto_now_add=True)</code>	

Relationships. 1-many to `DoctorProfile` (`on_delete=PROTECT` — a specialty in use cannot be hard-deleted).

Indexes. `name`, `slug` (unique, implicit); `is_active` for the “active specialties” filter used on every doctor-search request.

4.3.4 `DoctorProfile` (app: `doctors`)

Purpose. Doctor-specific data, one row per doctor `User`.

Field	Type	Notes
<code>user</code>	<code>OneToOneField(User, primary_key=True, on_delete=CASCADE)</code>	
<code>specialty</code>	<code>ForeignKey(Specialty, on_delete=PROTECT)</code>	
<code>license_number</code>	<code>CharField(50, unique=True)</code>	verified manually by admin before approval
<code>years_of_experience</code>	<code>PositiveSmallIntegerField</code>	
<code>consultation_fee</code>	<code>DecimalField(max_digits=8, decimal_places=2, null=True, blank=True)</code>	informational only for MVP (2.9, Assumption 8)
<code>bio</code>	<code>TextField(blank=True)</code>	
<code>clinic_address</code>	<code>TextField(blank=True)</code>	
<code>profile_picture</code>	<code>ImageField(upload_to='doctors/', null=True, blank=True)</code>	see Section 9.10 secure upload rules
<code>is_accepting_appointments</code>	<code>BooleanField(default=True)</code>	doctor-controlled toggle, independent of admin <code>account_status</code>

Field	Type	Notes
created_at, updated_at	DateTimeField	

Relationships. 1-1 to User; many-1 to Specialty; 1-many to DoctorAvailability, Appointment (as doctor).

Indexes. specialty_id (FK index, auto), composite (specialty_id, is_accepting_appointments) for search.

Validators. license_number format validated against a configurable regex per Section 8; profile only creatable for a User with role= 'DOCTOR' (CheckConstraint is not expressible across tables in plain Django, so this is enforced in services.py at creation time and covered by a dedicated unit test).

4.3.5 PatientProfile (app: patients)

Purpose. Patient-specific data, one row per patient User.

Field	Type	Notes
user	OneToOneField(User, primary_key=True, on_delete=CASCADE)	
date_of_birth	DateField(null=True, blank=True)	
gender	CharField(20, choices=Gender, blank=True)	
address	TextField(blank=True)	
emergency_contact_name	CharField(150, blank=True)	
emergency_contact_phone	CharField(20, blank=True)	
blood_type	CharField(5, blank=True)	
profile_picture	ImageField(upload_to='patients/', null=True, blank=True)	
created_at, updated_at	DateTimeField	

Relationships. 1-1 to User; 1-many to Appointment (as patient).

Indexes. None beyond the PK — this table is only ever looked up by user_id.

Validators. date_of_birth must not be in the future.

4.3.6 DoctorAvailability (app: doctors)

Purpose. A recurring weekly template a doctor defines once; bookable slots are computed from it at request time (Section 4.6), not materialized into a separate slots table for MVP.

Field	Type	Notes
id	BigAutoField	PK
doctor	ForeignKey(DoctorProfile, on_delete=CASCADE, related_name='availabilities')	
day_of_week	SmallIntegerField(choices=Weekday)	0=Monday ... 6=Sunday
start_time	TimeField	
end_time	TimeField	
slot_duration_minutes	PositiveSmallIntegerField(default=30)	
is_active	BooleanField(default=True)	

Relationships. Many-1 to DoctorProfile.

Indexes. Composite (doctor_id, day_of_week, is_active) — the exact lookup used to compute a doctor's slots for a given date.

Constraints. CheckConstraint(start_time < end_time); UniqueConstraint(doctor, day_of_week, start_time, end_time) to prevent literal duplicate rows (overlap *within* a day beyond exact duplicates is rejected at the serializer level — see 8.6 — because expressing “no overlapping time ranges” as a portable DB constraint is disproportionately complex for MVP).

Validators. (end_time - start_time) must be evenly divisible by slot_duration_minutes (serializer-level).

4.3.7 Appointment (app: appointments)

Purpose. The core transactional record of the system and the table with the strictest integrity requirements.

Field	Type	Notes
id	BigAutoField	PK
patient	ForeignKey(PatientProfile, on_delete=PROTECT, related_name='appointments')	PROTECT, not CASCADE — a patient record must never be hard-deleted while appointment history references it

Field	Type	Notes
	ments')	
doctor	ForeignKey(DoctorProfile, on_delete=PROTECT, related_name='appointments')	same rationale
appointment_date	DateTimeField	
start_time, end_time	TimeField	derived from the doctor's availability at booking time
status	CharField(15, choices=AppointmentStatus)	PENDING, CONFIRMED, REJECTED, CANCELLED, COMPLETED, RESCHEDULED
reason	TextField(blank=True)	patient-supplied reason for visit
doctor_notes	TextField(blank=True)	doctor-supplied, visible to patient once set
cancellation_reason	TextField(blank=True)	
cancelled_by	ForeignKey(User, null=True, blank=True, on_delete=SET_NULL, related_name='+')	who initiated the cancellation
previous_appointment	ForeignKey('self', null=True, blank=True, on_delete=SET_NULL, related_name='rescheduled_to')	links a new booking back to the one it replaced
created_at, updated_at	DateTimeField	

Relationships. Many-1 to PatientProfile, DoctorProfile, User (cancelled_by), self (previous_appointment); 1-many to Notification.

Indexes. (doctor_id, appointment_date), (patient_id, appointment_date), (status) — these three cover every list/filter query in Section 6.

Constraints — the load-bearing one.

```
class Meta:
    constraints = [
        models.UniqueConstraint(
            fields=["doctor", "appointment_date", "start_time"],
            condition=~models.Q(status__in=["CANCELLED", "REJECTED"])),
```

```

        name="uniq_active_doctor_slot",
    ),
    models.CheckConstraint(
        check=models.Q(start_time__lt=models.F("end_time")),
        name="appointment_start_before_end",
    ),
]

```

This partial unique index means the database itself refuses a second active row for the same doctor/date/start_time — cancelled and rejected rows are excluded from the constraint so a freed slot can be rebooked. This is the single sentence in this document that most directly satisfies FR-17 and mitigates Risk R-1.

Validators. `appointment_date >= today` on create (serializer); `start_time/end_time` must exactly match a slot derived from the doctor's `DoctorAvailability` (service-level — see 4.6 and Section 7.6); status transitions validated by a state-transition table, not left open (Section 7.7).

4.3.8 Notification (app: notifications)

Purpose. Durable log of every outbound email — both the audit trail Risk R-2 needs and the retry queue Celery consumes.

Field	Type	Notes
<code>id</code>	<code>BigAutoField</code>	PK
<code>recipient</code>	<code>ForeignKey(User, on_delete=CASCADE, related_name='notifications')</code>	
<code>related_appointment</code>	<code>ForeignKey(Appointment, null=True, blank=True, on_delete=SET_NULL)</code>	
<code>notification_type</code>	<code>CharField(30, choices=NotificationType)</code>	EMAIL_VERIFICATION, PASSWORD_RESET, BOOKING_CONFIRMATION, BOOKING_CANCELLED, BOOKING_RESCHEDULED, DOCTOR_APPROVED, DOCTOR_REJECTED, APPOINTMENT_REMINDER
<code>status</code>	<code>CharField(10, choices=['PENDING', 'SENT', 'FAILED'])</code>	
<code>subject</code>	<code>CharField(255)</code>	
<code>body</code>	<code>TextField</code>	rendered HTML/text body, stored

Field	Type	Notes
<code>error_message</code>	<code>TextField(blank=True)</code>	for audit even after sending populated on failure
<code>sent_at</code>	<code>DateTimeField(null=True, blank=True)</code>	
<code>created_at</code>	<code>DateTimeField(auto_now_add=True)</code>	

Relationships. Many-1 to User, Appointment.

Indexes. (`status`, `created_at`) — the exact query the retry worker uses to find failed sends.

4.3.9 AuditLog (app: accounts)

Purpose. Immutable record of admin actions with state-changing effect on another user, satisfying the “Auditability” non-functional requirement (2.2) and Risk R-3.

Field	Type	Notes
<code>id</code>	<code>BigAutoField</code>	PK
<code>actor</code>	<code>ForeignKey(User, null=True, on_delete=SET_NULL, related_name='+')</code>	null only for system-initiated actions
<code>target_user</code>	<code>ForeignKey(User, null=True, on_delete=SET_NULL, related_name='+')</code>	
<code>action</code>	<code>CharField(50)</code>	e.g., DOCTOR_APPROVED, DOCTOR_BLOCKED, PATIENT_BLOCKED
<code>description</code>	<code>TextField(blank=True)</code>	
<code>metadata</code>	<code>JSONField(default=dict, blank=True)</code>	free-form structured detail (e.g., affected appointment IDs for R-3)
<code>created_at</code>	<code>DateTimeField(auto_now_add=True)</code>	

Relationships. Many-1 to User (twice, as actor and target).

Indexes. (`target_user`, `created_at`), (`actor`, `created_at`).

Constraints. Application-enforced immutability — no update/delete endpoint is ever exposed for this model; only Django Admin’s read-only list view and the service layer’s `create()`.

4.4 Relationship Summary

- `User` is the identity root; `DoctorProfile`/`PatientProfile` are strict 1:1 extensions selected by `role`.
- `Appointment` is the hub of the domain: every other new feature (reviews, prescriptions, billing — Section 16) will hang off it.
- `previous_appointment` is a self-referential FK forming a linked list of reschedules, so `appointment.rescheduled_to.first()` always finds the live booking that replaced a given one.
- `PROTECT` is used on the two FKs (`patient`, `doctor`) that must never silently disappear out from under appointment history; `CASCADE` is reserved for true ownership (a `User` deleted removes their own `DoctorProfile`/`PatientProfile`, but not their appointments, which are protected one level down).

4.5 Database Optimization Recommendations

- All FK columns get a DB index automatically via Django; the composite indexes listed per-model above are additional and must be added explicitly via `Meta.indexes`.
- `appointment_date` range queries (“doctor’s appointments this week”) benefit from PostgreSQL BRIN indexing only at much larger scale than MVP — start with a plain B-tree composite index and revisit under Section 16 if the table passes ~10M rows.
- Use `select_related('doctor__specialty', 'patient')` on every `Appointment` list endpoint to avoid N+1 queries — this is a code-review checklist item (Section 11).
- Cache `Specialty` list (rarely changes) in Redis with a short TTL and an explicit cache-bust on admin write, since it is read on nearly every doctor-search request.
- Consider a materialized “generated slots” table only if/when slot computation (Section 4.6) becomes a measured bottleneck — do not build it pre-emptively.

4.6 Deriving Bookable Slots (Not a Table, a Computation)

For a given doctor and date, the bookable slots are computed, not stored:

1. Look up the doctor’s active `DoctorAvailability` row(s) for that date’s `day_of_week`.
2. Generate candidate slots by walking `start_time` → `end_time` in steps of `slot_duration_minutes`.
3. Subtract any slot that overlaps an existing `Appointment` for that doctor/date with `status` not in `{CANCELLED, REJECTED}`.
4. Subtract any slot whose start is in the past relative to “now” (for `appointment_date == today`).

This keeps the database from ever holding a stale “available” flag that must be kept in sync with two other tables — the availability list and the appointment list are the only sources of truth.

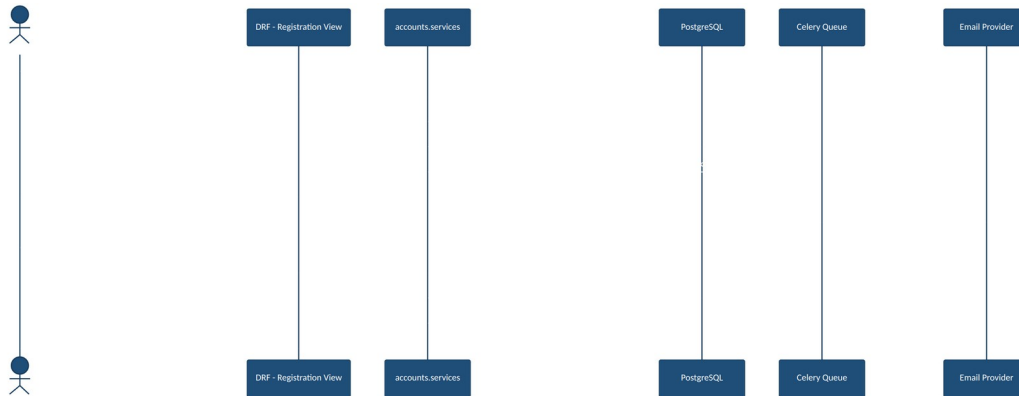
4.7 Future Scalability Considerations

- **Read replicas** for doctor-search traffic once write load (bookings) and read load (search) need to scale independently.
- **Partition Appointment by appointment_date** (range partitioning) once historical volume makes full-table scans expensive even with indexes.
- **Materialize generated slots** into a real table with its own uniqueness constraint if slot computation ever shows up in profiling — trades write complexity (must regenerate on availability change) for read speed.
- **Multi-specialty doctors** would require a many-to-many `DoctorProfile ↔ Specialty` instead of the MVP's one-to-many — flagged here so the FK is not assumed unchangeable elsewhere in the codebase.
- **Full-text search** (doctor name, bio) via PostgreSQL `tsvector` if simple `icontains` filtering (Section 6) stops being fast enough.

5. Authentication & Authorization

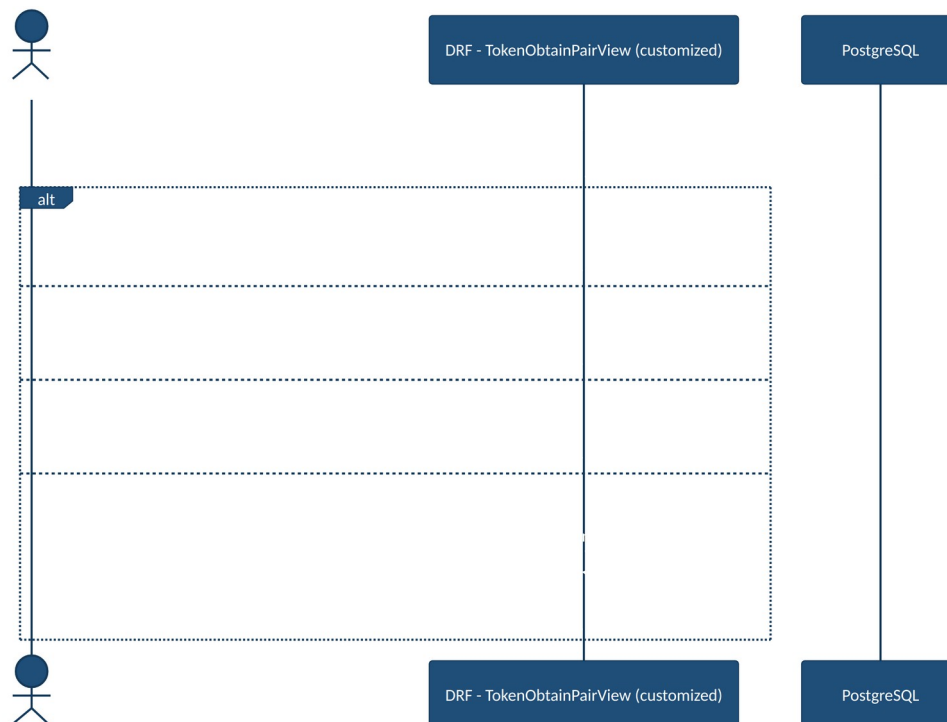
5.1 Registration Flow

Patients and doctors both register through the same endpoint with a `role` field; the account is created immediately but is unusable until email verification (and, for doctors, admin approval) completes.



Doctor accounts land in `account_status=PENDING_APPROVAL` regardless of email verification, and cannot log in until an admin approves them (5.9). Patient accounts land in `account_status=ACTIVE` immediately upon email verification.

5.2 Login Flow



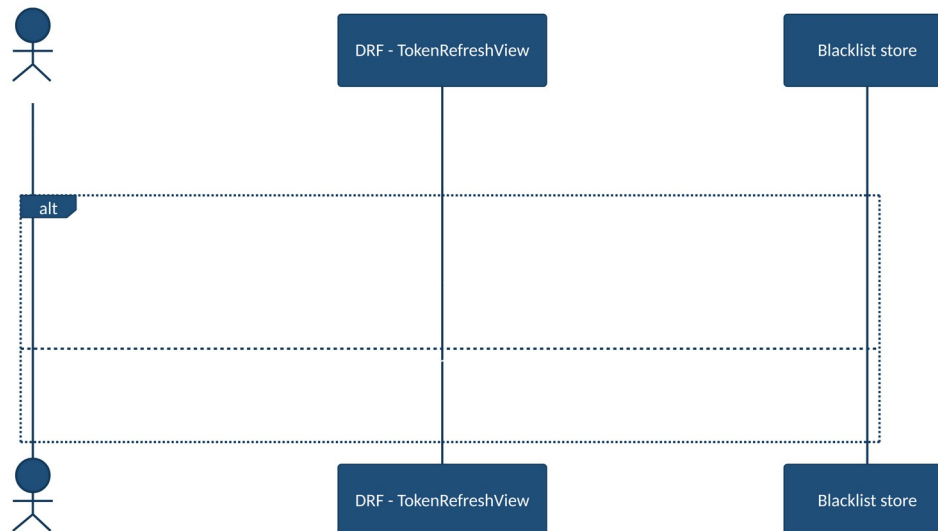
The default `SimpleJWT TokenObtainPairView` is subclassed to inject these pre-checks before token issuance, rather than allowing a valid-password-but-unverified user to receive tokens and get rejected later on every subsequent call.

5.3 JWT Lifecycle

Token	Lifetime	Notes
Access token	15 minutes	Sent as Authorization: Bearer <token> ; short-lived to limit blast radius of leakage
Refresh token	7 days	Rotated on every use (<code>ROTATE_REFRESH_TOKENS=True</code>); old token blacklisted (<code>BLACKLIST_AFTER_ROTATION=True</code>)

`rest_framework_simplejwt.token_blacklist` app is enabled so logout and rotation are enforceable server-side, not just a client-side token discard.

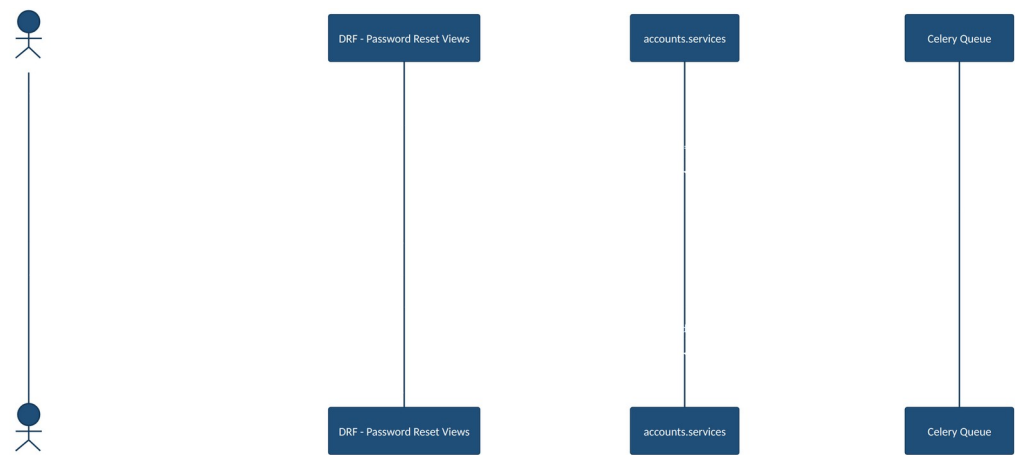
5.4 Refresh Tokens



5.5 Email Verification

- Endpoint: `GET /api/v1/auth/verify-email/{token}/`.
- Token expiry: 24 hours (2.4, Hidden Requirements).
- Resend endpoint: `POST /api/v1/auth/resend-verification/` — invalidates any prior unused token for that user and issues a new one, rate-limited to 1 request/minute per account to prevent email-bombing.
- Login is blocked with a specific `EMAIL_NOT_VERIFIED` error code (not a generic 401) so the frontend can show a “resend verification email” action.

5.6 Password Reset



The confirmation step always responds 200 OK on the *request* step regardless of whether the email exists, to avoid leaking account existence — a standard OWASP recommendation (Section 9).

5.7 Permissions

Permission class	Grants
IsAuthenticated	Base — any logged-in, active, verified user (DRF built-in)
IsAdmin	<code>request.user.role == 'ADMIN'</code>
IsDoctor	<code>request.user.role == 'DOCTOR'</code> and <code>account_status == 'ACTIVE'</code>
IsPatient	<code>request.user.role == 'PATIENT'</code> and <code>account_status == 'ACTIVE'</code>
IsAdminOrReadOnly	Admin can write; any authenticated user can read (used on <code>Specialty</code>)
IsOwnerPatientOrAdmin	Object-level: the <code>Appointment.patient.user == request.user</code> , or Admin
IsOwnerDoctorOrAdmin	Object-level: the <code>Appointment.doctor.user == request.user</code> , or Admin

5.8 Role-Based Access Summary

Resource	Patient	Doctor	Admin
Own profile	Read/Update	Read/Update	Read/Update
Other users' profiles	X	X (except patients they have an appointment with — read-only)	Read/Update (approve/block)

Resource	Patient	Doctor	Admin
		name/contact)	
Specialties	Read	Read	Full CRUD
Doctor availability	Read (any doctor)	Full CRUD (own only)	Read/Update any
Appointments — create	Create (own, as patient)	✗	✗
Appointments — read	Own only	Own only (as doctor)	All
Appointments — cancel	Own, PENDING/CONFIRME D only	✗ (doctor rejects, doesn't cancel)	Any
Appointments — approve/reject/comple te	✗	Own only	Any
Django Admin site	✗	✗	Full access

5.9 Object-Level Permissions

Row-level checks are implemented via DRF's `has_object_permission`, not by filtering querysets alone, so that an authorization bug in a queryset filter cannot be silently masked — every detail/update/delete view enforces the check twice defense-in-depth style: the queryset is scoped to what the user *should* see, and the object permission re-verifies ownership on the specific instance. Example: a patient hitting `GET /appointments/{id}/` for another patient's appointment gets **404 Not Found** (not 403), so the endpoint does not leak the existence of other patients' appointment IDs.

5.10 Security Considerations for This Section

- Passwords hashed with Django's default (PBKDF2SHA256); `AUTH_PASSWORD_VALIDATORS` enforces minimum length (12) and rejects common passwords.
- All auth endpoints are rate-limited (`django-ratelimit` or DRF throttling) — login: 5/min/IP, registration: 3/hour/IP, password-reset-request: 3/hour/account.
- Refresh tokens are rotated and blacklisted on use; a stolen refresh token that gets used once by an attacker invalidates itself for the legitimate user too, which surfaces the compromise (forced re-login) rather than hiding it.
- `SECRET_KEY` and JWT signing key are environment variables, never defaults, never committed (Section 9.11).
- See Section 9 for the full security posture (CORS, CSRF, XSS, injection, secrets management).

6. REST API Design

6.1 Conventions

- **Base path & versioning:** all endpoints are prefixed `/api/v1/`. Breaking changes ship as `/api/v2/` running alongside `v1` for a deprecation window; non-breaking additions (new optional fields, new endpoints) do not bump the version.
- **Authentication: Authorization:** Bearer `<access_token>` header, except the four public auth endpoints (register, login, refresh, password-reset-request) and `GET /specialties/`, `GET /doctors/` (public browse, per product requirement “view list of available doctors” — read-only, no PII beyond what’s already public-facing).
- **Content type:** `application/json` for all request/response bodies except profile picture uploads (`multipart/form-data`).
- **Resource naming:** plural, kebab-case nouns; nested actions use a trailing verb sub-path (`/appointments/{id}/cancel/`) rather than overloading `PATCH` with a `status` field the client has to know the state machine to use correctly.

6.2 Standard Response Format

Every successful response is wrapped in a consistent envelope so the frontend can write one generic response handler:

```
{
  "success": true,
  "data": { },
  "meta": { }
}
```

`meta` carries pagination info on list endpoints (6.5) and is omitted (or empty) on detail endpoints.

6.3 Standard Error Format

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "One or more fields are invalid.",
    "details": {
      "start_time": ["This slot is no longer available."]
    }
  }
}
```

`code` is a stable machine-readable string the frontend can switch on (e.g., to show a specific inline message for `EMAIL_NOT_VERIFIED` vs. a generic toast for `VALIDATION_ERROR`); `message` is human-readable; `details` is present only for field-level validation errors and mirrors DRF’s native serializer error

shape. This is implemented once, centrally, via a custom DRF exception handler in `core/exceptions.py` — individual views never hand-format error bodies.

6.4 Standard Status Codes

Code	Meaning in this API
200	Success (read, update, or action endpoint)
201	Resource created
204	Success, no body (e.g., delete)
400	Validation error (<code>details</code> populated)
401	Missing/invalid/expired credentials
403	Authenticated but not permitted (includes <code>EMAIL_NOT_VERIFIED</code> , <code>ACCOUNT_PENDING</code> , <code>ACCOUNT_BLOCKED</code>)
404	Not found, or found but not visible to this user (object-level permission, Section 5.9)
409	Conflict with current state (e.g., slot just taken, invalid status transition)
422	Semantically invalid but well-formed (rare; 400 covers most cases)
429	Rate limited
500	Unhandled server error (logged + Sentry, generic body returned to client)

6.5 Pagination

All list endpoints use DRF `PageNumberPagination` with a default page size of 20 (max 100 via `?page_size=`):

```
{
  "success": true,
  "data": [ ],
  "meta": {
    "pagination": {
      "count": 137,
      "page": 1,
      "page_size": 20,
      "total_pages": 7,
      "next": "/api/v1/doctors/?page=2",
      "previous": null
    }
  }
}
```

6.6 Filtering, Searching, Sorting

Implemented via `django-filter` + DRF `SearchFilter/OrderingFilter`, declared per-view, never ad-hoc query-param parsing:

Endpoint	Filter params	Search (?search=)	Ordering (?ordering=)
GET <code>/doctors/</code>	specialty, is_accepting_appointments	first_name, last_name, specialty__name	years_of_experience, consultation_fee
GET <code>/appointments/</code>	status, doctor, patient, date_from, date_to	—	appointment_date, created_at (default: - appointment_date)
GET <code>/users/</code> (admin)	role, account_status	email, first_name, last_name	date_joined
GET <code>/notifications/</code> (admin)	status, notification_type	—	created_at

6.7 API Versioning

`v1` is the only version at launch. The version lives in the URL path (`/api/v1/...`), not in a header, so it is visible in logs and trivially routable at the reverse proxy if `v1` and `v2` ever need to be split across deployments. `drf-spectacular` generates the OpenAPI schema per version from the DRF views/serializers themselves, so the schema in `docs/openapi/` can never silently drift from the implementation — it is regenerated in CI and diffed against the committed copy.

6.8 Endpoint Catalogue

Every endpoint below follows the standard response/error format (6.2, 6.3) and status codes (6.4). Full request/response examples for the highest-complexity endpoints follow in 6.9.

6.8.1 Authentication (/api/v1/auth/)

Method	URL	Auth	Description	Success	Key errors
POST	/auth/register/	Public	Create Patient or Doctor account	201	400 (validation, duplicate email)
GET	/auth/verify-email/{token}/	Public	Confirm email via emailed link	200	400 TOKEN_INVALID_OR_EXPIRED
POST	/auth/resend-verification/	Public	Reissue verification email	200	429 (rate-limited)
POST	/auth/login/	Public	Obtain JWT pair	200	401, 403 (EMAIL_NOT_VERIFIED/ACCOUNT_PENDING/ACCOUNT_BLOCKED)
POST	/auth/refresh/	Public (refresh token in body)	Rotate access/refresh pair	200	401
POST	/auth/logout/	Authenticated	Blacklist current refresh token	204	400
POST	/auth/password-reset/	Public	Request reset email	200 (always)	—
POST	/auth/password-reset-confirm/	Public	Set new password via token	200	400 TOKEN_INVALID_OR_EXPIRED

6.8.2 Accounts / Users (/api/v1/users/)

Method	URL	Auth	Permission	Description	Success	Key errors
GET	/users/me/	Authenticated	Self	Current user's base	200	401

Method	URL	Auth	Permission	Description	Success	Key errors
PATCH	/users/me/	Authenticated	Self	account data Update name/phone (not email/role)	200	400
POST	/users/change-password/	Authenticated	Self	Change password (requires current password)	200	400
GET	/users/	Authenticated	Admin	List all users, filterable	200	403
GET	/users/{id}/	Authenticated	Admin	User detail	200	403, 404
PATCH	/users/{id}/approve/	Authenticated	Admin	Approve pending doctor/patient (account_status → ACTIVE)	200	403, 404, 409 (already active)
PATCH	/users/{id}/block/	Authenticated	Admin	Block a user (account_status → BLOCKED)	200	403, 404
PATCH	/users/{id}/unblock/	Authenticated	Admin	Restore to ACTIVE	200	403, 404

6.8.3 Specialties (/api/v1/specialties/)

Method	URL	Auth	Permission	Description	Success	Key errors
GET	/specialties/	Public	—	List active specialties	200	—
POST	/specialties/	Authenticated	Admin	Create specialty	201	400, 403
GET	/specialty	Public	—	Specialty detail	200	404

Method	URL	Auth	Permission	Description	Success	Key errors
PATCH	ies/ {id}/ /specialties/ {id}/	Authenticated	Admin	Update specialty	200	400, 403, 404
DELETE	/specialties/ {id}/	Authenticated	Admin	Soft-delete (is_active=False)	204	403, 404, 409 (doctors still assigned — warns, does not block)

6.8.4 Doctors (/api/v1/doctors/)

Method	URL	Auth	Permission	Description	Success	Key errors
GET	/doctors/	Public	—	Browse/filter doctors (6.6)	200	—
GET	/doctors/{id}/	Public	—	Doctor profile detail	200	404
GET	/doctors/me/	Authenticated	Doctor	Own profile (includes fields not shown publicly)	200	403
PATCH	/doctors/me/	Authenticated	Doctor	Update own profile	200	400, 403
GET	/doctors/{id}/slots/?date=	Public	—	Computed bookable slots for a date (4.6)	200	400 (bad date), 404
GET	/doctors/me/availabilities/	Authenticated	Doctor	List own weekly availability rows	200	403
POST	/doctors/	Authenticated	Doctor	Create an availability	201	400, 403

Method	URL	Auth	Permission	Description	Success	Key errors
	me/availabilities/			window		
PATCH	/doctors/me/availabilities/{id}/	Authenticated	Doctor	Update a window	200	400, 403, 404
DELETE	/doctors/me/availabilities/{id}/	Authenticated	Doctor	Remove a window	204	403, 404

6.8.5 Patients (/api/v1/patients/)

Method	URL	Auth	Permission	Description	Success	Key errors
GET	/patients/me/	Authenticated	Patient	Own profile	200	403
PATCH	/patients/me/	Authenticated	Patient	Update own profile	200	400, 403
GET	/patients/{id}/	Authenticated	Doctor (shared appointment only) / Admin	Limited patient view	200	403, 404

6.8.6 Appointments (/api/v1/appointments/)

Method	URL	Auth	Permission	Description	Success	Key errors
POST	/appointments/	Authenticated	Patient	Book a slot	201	400, 409 SLOT_UNAVAILABLE
GET	/appointments/	Authenticated	Patient/Doctor (own)/Admin (all)	List, role-scoped, filterable (6.6)	200	401
GET	/appointments/{id}/	Authenticated	Owner (patient or doctor)	Detail	200	404

Method	URL	Auth	Permission	Description	Success	Key errors
POST	ents/{id}/ /appointments/{id}/cancel/	Authenticate d	doctor) / Admin Owning patient / Admin	Cancel (PENDING/CONFIRMED only)	200	409 (invalid state), 404
POST	/appointments/{id}/reschedule/	Authenticate d	Owning patient	Cancel + create new linked appointment	201	400, 409
POST	/appointments/{id}/approve/	Authenticate d	Owning doctor / Admin	PENDING → CONFIRMED	200	409 (invalid state), 404
POST	/appointments/{id}/reject/	Authenticate d	Owning doctor / Admin	PENDING → REJECTED (requires reason)	200	400, 409, 404
POST	/appointments/{id}/complete/	Authenticate d	Owning doctor / Admin	CONFIRMED → COMPLETE D	200	409 (invalid state), 404

6.8.7 Notifications & Audit (admin visibility)

Method	URL	Auth	Permission	Description	Success	Key errors
GET	/notifications/	Authenticate d	Admin	List sent/failed notifications, filterable	200	403
GET	/audit-logs/	Authenticate d	Admin	Read-only admin action history	200	403

6.9 Detailed Request/Response Examples

6.9.1 POST /api/v1/auth/register/

Request

```
{
  "email": "sara.patient@example.com",
  "password": "Str0ngPass!2026",
  "password_confirm": "Str0ngPass!2026",
  "first_name": "Sara",
  "last_name": "Ahmed",
  "phone_number": "+201001234567",
  "role": "PATIENT"
}
```

Response — 201 Created

```
{
  "success": true,
  "data": {
    "id": 42,
    "email": "sara.patient@example.com",
    "role": "PATIENT",
    "account_status": "ACTIVE",
    "is_email_verified": false
  },
  "meta": {}
}
```

Validation rules: email unique, valid format; password/password_confirm must match and pass AUTH_PASSWORD_VALIDATORS; role must be PATIENT or DOCTOR (never ADMIN via this endpoint — enforced server-side regardless of what the client sends); phone_number matches the configured regex. Doctor registrations additionally require specialty_id and license_number, which create the linked DoctorProfile in the same transaction and set account_status=PENDING_APPROVAL.

Errors: 400 with details.email = ["A user with this email already exists."];
400 with details.password_confirm = ["Passwords do not match."].

6.9.2 POST /api/v1/auth/login/

Request

```
{ "email": "sara.patient@example.com", "password": "Str0ngPass!2026" }
```

Response — 200 OK


```
{
  "success": true,
  "data": {
    "access": "eyJhbGciOiJIUzI1NiIs... ",
    "refresh": "eyJhbGciOiJIUzI1NiIs... ",
    "user": { "id": 42, "email": "sara.patient@example.com", "role":
"PATIENT" }
  },
  "meta": {}
}
```

Response — 403 Forbidden (unverified email)

```
{
  "success": false,
  "error": {
    "code": "EMAIL_NOT_VERIFIED",
    "message": "Please verify your email address before logging in.",
    "details": {}
  }
}
```

6.9.3 GET /api/v1/doctors/{id}/slots/?date=2026-07-20

Response — 200 OK

```
{
  "success": true,
  "data": {
    "doctor_id": 7,
    "date": "2026-07-20",
    "slot_duration_minutes": 30,
    "slots": [
      { "start_time": "09:00", "end_time": "09:30", "is_available":
true },
      { "start_time": "09:30", "end_time": "10:00", "is_available":
false },
      { "start_time": "10:00", "end_time": "10:30", "is_available":
true }
    ]
  },
  "meta": {}
}
```

Booked slots are included with `is_available: false` (rather than omitted) so the frontend can render a full, evenly-spaced schedule grid instead of reconstructing gaps client-side.

6.9.4 POST /api/v1/appointments/ (Booking — Patient)

Request

```
{
  "doctor_id": 7,
  "appointment_date": "2026-07-20",
  "start_time": "10:00",
  "reason": "Follow-up on blood pressure medication"
}
```

Response — 201 Created

```
{
  "success": true,
  "data": {
    "id": 981,
    "doctor": { "id": 7, "full_name": "Dr. Youssef Hany", "specialty":
"Cardiology" },
    "patient": { "id": 42, "full_name": "Sara Ahmed" },
    "appointment_date": "2026-07-20",
    "start_time": "10:00",
    "end_time": "10:30",
    "status": "PENDING",
    "reason": "Follow-up on blood pressure medication",
    "created_at": "2026-07-12T09:15:00Z"
  },
  "meta": {}
}
```

Response — 409 Conflict (slot taken between page-load and submit)

```
{
  "success": false,
  "error": {
    "code": "SLOT_UNAVAILABLE",
    "message": "This slot is no longer available. Please choose
another.",
    "details": {}
  }
}
```

See Section 7.5 for the transactional mechanics that make this 409 — rather than a silent double-booking — the guaranteed outcome under concurrent requests.

6.9.5 POST /api/v1/appointments/{id}/cancel/

Request

```
{ "cancellation_reason": "Schedule conflict" }
```

Response — 200 OK

```
{
  "success": true,
  "data": { "id": 981, "status": "CANCELLED", "cancelled_by": 42 },
  "meta": {}
}
```

Response — 409 Conflict

```
{
  "success": false,
  "error": {
    "code": "INVALID_STATE_TRANSITION",
    "message": "Only pending or confirmed appointments can be
cancelled.",
    "details": {}
  }
}
```

6.9.6 POST /api/v1/appointments/{id}/reschedule/

Request

```
{ "appointment_date": "2026-07-22", "start_time": "11:00" }
```

Response — 201 Created

```
{
  "success": true,
  "data": {
    "id": 1005,
    "status": "PENDING",
    "appointment_date": "2026-07-22",
    "start_time": "11:00",
    "end_time": "11:30",
    "previous_appointment_id": 981
  },
  "meta": {}
}
```

The original appointment (981) transitions to RESCHEDULED in the same DB transaction that creates 1005 — see 7.9.

6.9.7 POST /api/v1/appointments/{id}/reject/ (Doctor)

Request

```
{ "cancellation_reason": "Requested time no longer available due to an  
emergency." }
```

Response — 200 OK

```
{  
  "success": true,  
  "data": { "id": 981, "status": "REJECTED" },  
  "meta": {}  
}
```

`cancellation_reason` (renamed conceptually “doctor’s reason” in the frontend copy, same field) is **required** on reject — enforced at the serializer level — because a rejection with no reason is the single most common source of patient support tickets (2.4).

7. Business Logic

This section specifies the internal workflows behind the endpoints in Section 6. Each is implemented as a function or class in the relevant app's `services.py`, called from a thin view — never implemented inline in a view or a serializer's `save()`.

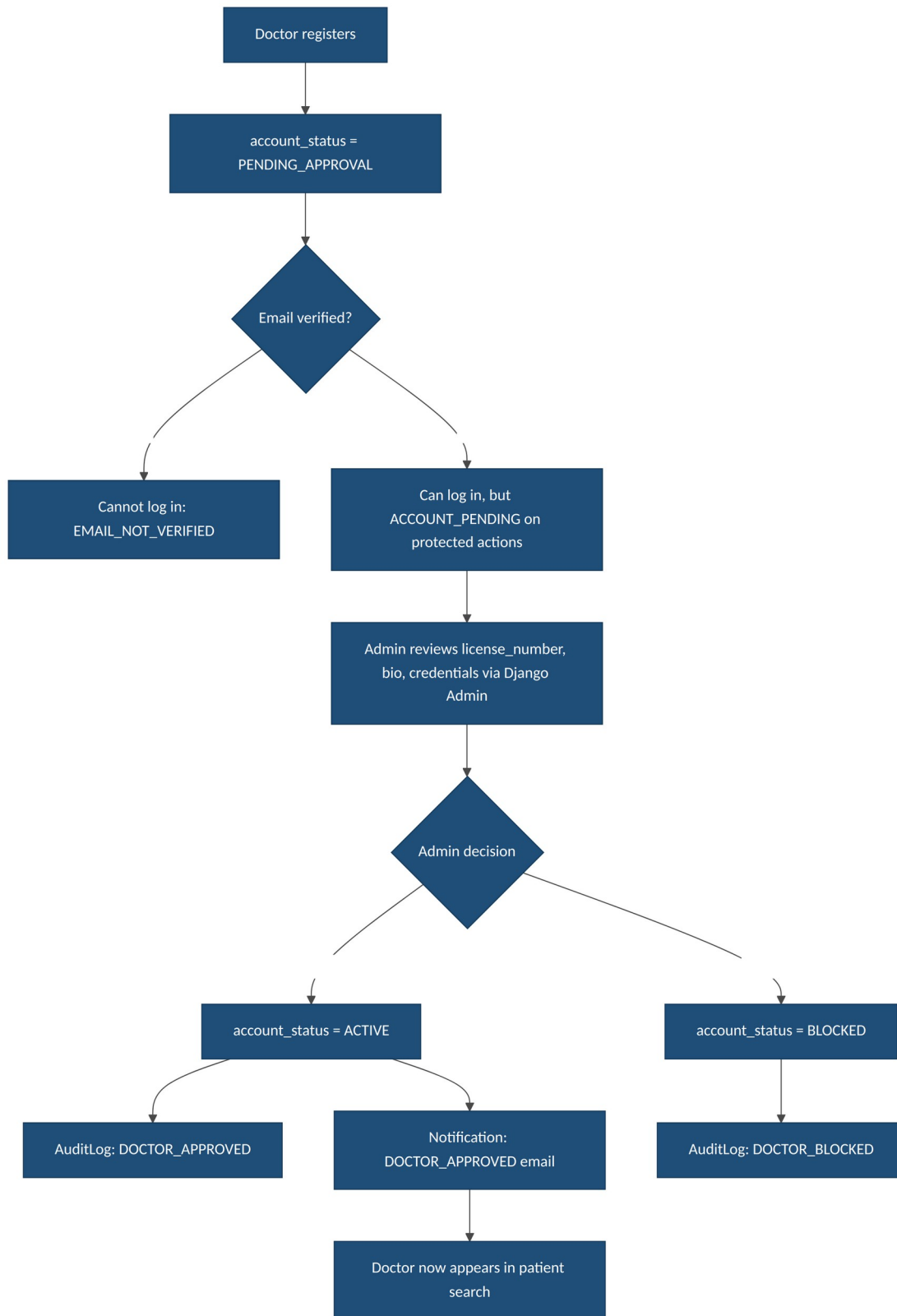
7.1 Registration

`accounts.services.register(data):`

1. Validate uniqueness of `email`.
2. Create `User` with `role` from input, `is_email_verified=False`.
3. If `role == DOCTOR`: set `account_status = PENDING_APPROVAL`, create the linked `DoctorProfile` from `specialty_id/license_number` in the same DB transaction.
4. If `role == PATIENT`: set `account_status = ACTIVE`, create the linked `PatientProfile` (empty, filled in later via `PATCH /patients/me/`).
5. Create a `VerificationToken` (purpose `EMAIL_VERIFICATION`, 24h expiry).
6. Enqueue `notifications.tasks.send_verification_email_task(user_id, token)`.
7. Return the created user — no JWTs issued (5.1).

All of steps 2–5 run inside `transaction.atomic()`: a failure creating the `DoctorProfile` must roll back the `User` row, or the system ends up with an unusable orphan account.

7.2 Doctor Approval



`accounts.services.approve_user(admin, target_user)` and `block_user(admin, target_user, reason)` both: (a) update `account_status`, (b) write an `AuditLog` row with `actor=admin`, `target_user=target_user`, (c) for `block_user`, first compute the count of the doctor's future `PENDING/CONFIRMED` appointments and include it in the `AuditLog.metadata` and in the API response as a warning (`{"affected_future_appointments": 4}`) — addressing Risk R-3 without auto-cancelling anything, per the conservative decision in Section 2.4.

7.3 Patient Management

Patient self-service is intentionally simple: a patient can view and update their own `PatientProfile` (`PATCH /patients/me/`) and their base account fields (`PATCH /users/me/`). Admins can view any patient (`GET /users/{id}/`) and block/unblock but cannot edit patient profile *content* (medical fields) on their behalf for MVP — reducing the admin surface area and avoiding the question of who is liable for admin-entered medical data.

7.4 Doctor Availability Management

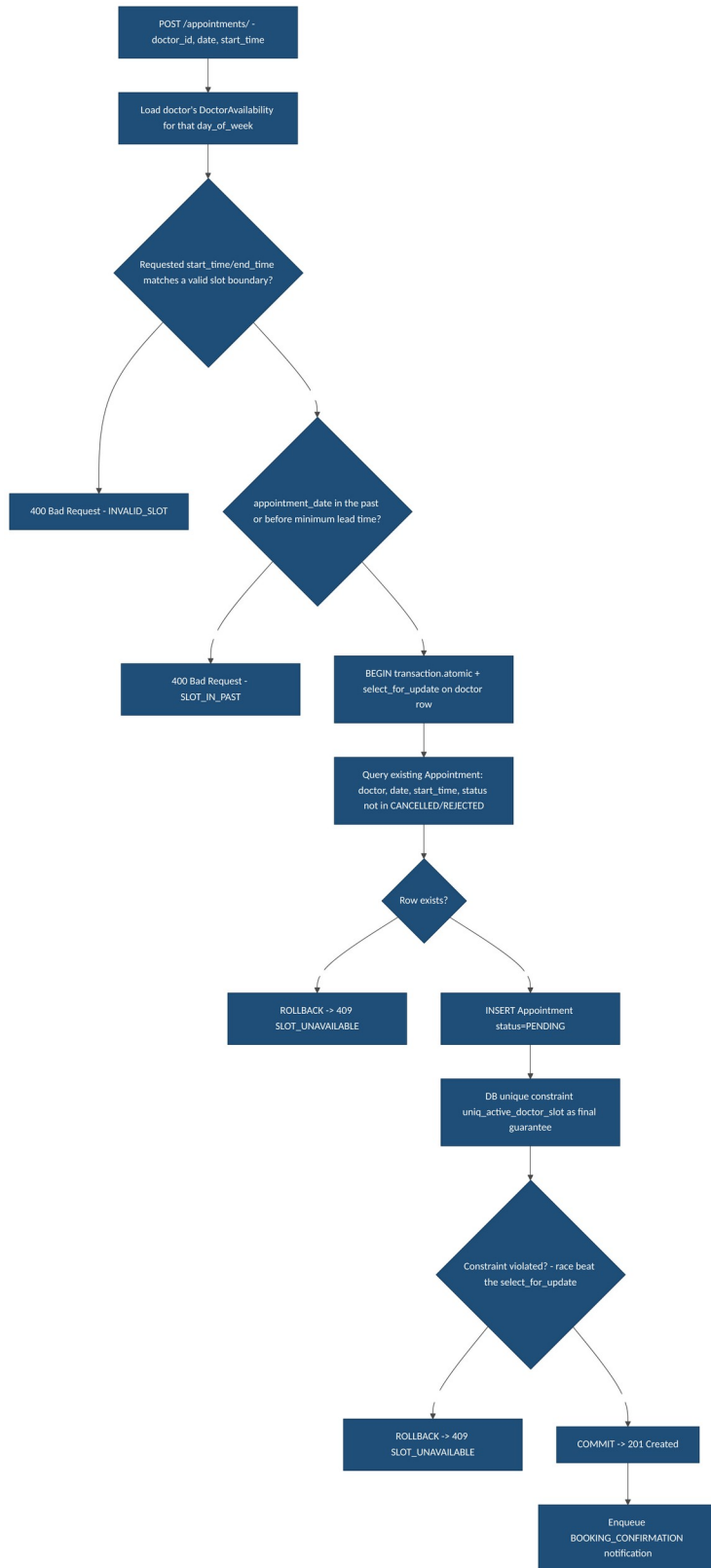
`doctors.services.create_availability(doctor, data):`

1. Validate `start_time < end_time` and that `(end_time - start_time) % slot_duration_minutes == 0`.
2. Validate no existing active `DoctorAvailability` row for the same `doctor + day_of_week` overlaps the requested window (application-level overlap check — 8.6).
3. Create the row.

Changing or deleting an availability window never touches existing `Appointment` rows (2.5, Edge Cases) — it only changes what `compute_available_slots()` (4.6) returns for *future* slot queries.

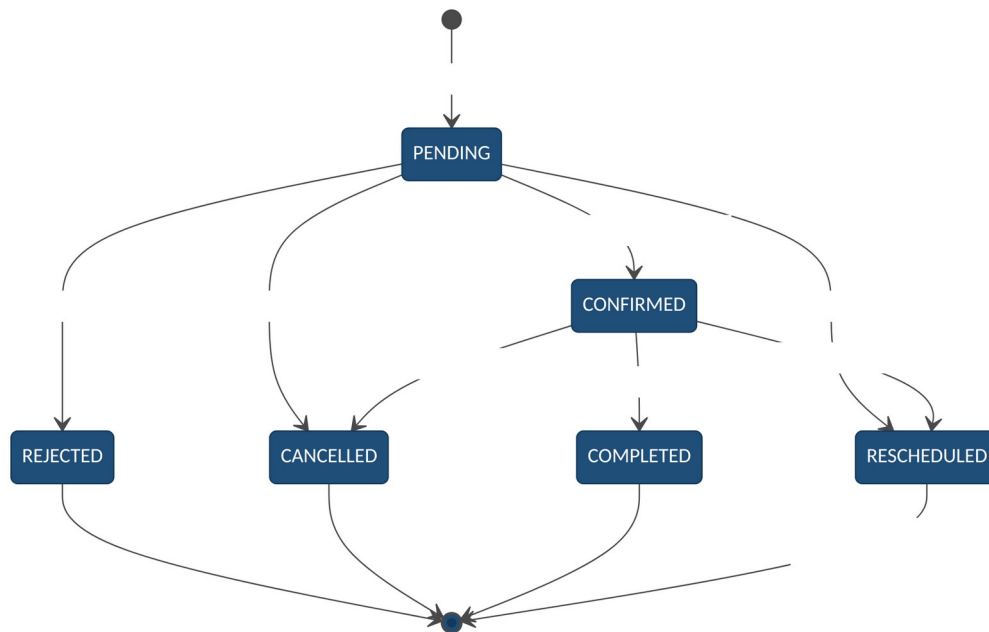
7.5 Booking & Double-Booking Prevention

This is the single most important piece of business logic in the system (FR-17, Risk R-1).



Two layers of protection are deliberate, not redundant: `select_for_update()` locks the doctor's availability for the duration of the transaction so concurrent requests serialize instead of racing, and the partial unique constraint (4.3.7) is the last line of defense that makes a double-booking a **database-level impossibility** even if application logic ever has a bug — this is the difference between “we tried to prevent it” and “it cannot happen.”

7.6 Appointment Lifecycle



Every transition above is the *only* legal transitions — enforced by an explicit table in `appointments/services.py` (not scattered `if` statements), so adding a future state (e.g., `NO_SHOW`, Section 16) means editing one table, not auditing every view.

7.7 Cancellation

`appointments.services.cancel_appointment(actor, appointment, reason):`

1. Check `appointment.status` in `{PENDING, CONFIRMED}` — else 409 `INVALID_STATE_TRANSITION`.
2. Check `actor` is the owning patient's user, or Admin — else 403.
3. Set `status = CANCELLED`, `cancelled_by = actor`, `cancellation_reason = reason`.
4. Enqueue `BOOKING_CANCELLED` notification to the doctor (and to the patient if an admin initiated it).

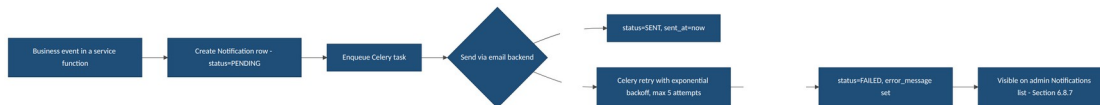
The slot is immediately bookable again because the partial unique constraint (4.3.7) excludes **CANCELLED** rows.

7.8 Rescheduling

`appointments.services.reschedule_appointment(patient, appointment, new_date, new_start_time)` runs the cancel logic and the booking logic (7.5) inside **one** `transaction.atomic()` block:

1. Validate `appointment.status` in `{PENDING, CONFIRMED}`.
2. Run the full booking validation (7.5) for the new date/time against the same doctor.
3. If the new slot is available: create the new `Appointment` with `previous_appointment = appointment, status = PENDING`; set the original appointment's `status = RESCHEDULED`.
4. If the new slot is unavailable: roll back everything — the **original appointment is untouched** (this is why it must be one transaction; a naive two-step “cancel then re-book” would leave the patient with nothing if the second step failed).

7.9 Notifications



The `Notification` row is written synchronously (inside the same transaction as the business event, e.g., booking) so that even if Celery/Redis is briefly unavailable, the audit record and retry queue survive a worker restart; only the actual send is asynchronous.

7.10 Admin Operations

Admins operate through both the API (Section 6.8.2, `approve/block`) and Django Admin (Section 3, for specialty/doctor CRUD and read-only browsing of all appointments/notifications/audit logs). Every admin action that mutates another user's state goes through the same service functions the API uses — Django Admin actions call `accounts.services.approve_user()` / `block_user()` directly rather than duplicating the logic, so the audit trail (7.2) is identical regardless of which surface the admin used.

8. Validation Rules

Validation is layered: DRF serializer field validation (format/type), serializer `validate()` (cross-field/business rule), and DB constraints (last line of defense, Section 4). This section documents the serializer/business layer per model.

8.1 User

Field	Required on create	Rules
email	Yes	Valid email format; unique (case-insensitive)
password	Yes	Min 12 chars; Django's common-password and similarity validators; must match <code>password_confirm</code>
first_name, last_name	Yes	1-150 chars, letters/spaces/hyphens only
phone_number	Yes	Regex <code>^\+?[1-9]\d{7,14}\$</code>
role	Yes	One of <code>PATIENT</code> , <code>DOCTOR</code> (never <code>ADMIN</code> via public registration)

Example invalid request: `{"email": "not-an-email"} → 400 {"details": {"email": ["Enter a valid email address."]}}`.

8.2 DoctorProfile

Field	Required	Rules
specialty_id	Yes (create)	Must reference an active <code>Specialty</code>
license_number	Yes (create)	Unique; matches configured format regex; immutable after creation
years_of_experience	No	0-70
consultation_fee	No	≥ 0, max 2 decimal places
bio	No	Max 2,000 chars
is_accepting_appointments	No	Boolean, defaults <code>True</code>

Business constraint: a `DoctorProfile` can only be created for a `User` with `role=DOCTOR` — enforced in `services.register()`, covered by a unit test asserting the reverse is rejected.

8.3 PatientProfile

Field	Required	Rules
date_of_birth	No	Must not be in the future; if provided, implies age ≥ 0
gender	No	One of the defined choices
emergency_contact_phone	No	Same regex as <code>User.phone_number</code>

8.4 Specialty

Field	Required	Rules
name	Yes	Unique (case-insensitive), 2–100 chars
slug	Auto	Generated from name; regenerated only if name changes and the new slug is still unique

Unique constraint: name, slug.

8.5 DoctorAvailability

Field	Required	Rules
day_of_week	Yes	0–6
start_time,end_time	Yes	<code>start_time < end_time</code>
slot_duration_minutes	Yes	One of 15, 20, 30, 45, 60; must evenly divide <code>end_time - start_time</code>

Business constraint (cross-row): the new window must not overlap any other **active** window for the same doctor and `day_of_week`. Checked in `validate()` against the database, not only against the `Meta.constraints` unique tuple, because exact-duplicate uniqueness alone does not catch partial overlaps (e.g., 09:00–12:00 and 11:00–14:00).

Example invalid request: `{"start_time": "09:00", "end_time": "12:10", "slot_duration_minutes": 30}` → 400 `{"details": {"slot_duration_minutes": ["190 minutes is not evenly divisible by 30."]}}`.

8.6 Appointment

Field	Required	Rules
doctor_id	Yes (create)	Must reference an ACTIVE , <code>is_accepting_appointments=True</code> doctor

Field	Required	Rules
appointment_date	Yes (create)	$\geq$ today; $\leq$ today + MAX_BOOKING_HORIZON_DAYS (default 90, configurable — 2.9)
start_time	Yes (create)	Must exactly match a slot boundary derived from the doctor's availability (7.5)
reason	No	Max 1,000 chars
cancellation_reason	Required when rejecting; optional when patient-cancelling	Max 1,000 chars
doctor_notes	No	Max 2,000 chars; settable only by the owning doctor or admin

Business constraints: - Only PENDING/CONFIRMED appointments may be cancelled or rescheduled (7.6). - Only PENDING appointments may be approved or rejected. - Only CONFIRMED appointments may be completed. - A patient may not have two PENDING/CONFIRMED appointments with the same doctor at overlapping times (defends against accidental double-submission from the frontend, distinct from the doctor-side double-booking constraint).

Unique constraint: uniq_active_doctor_slot (4.3.7, DB-level).

8.7 VerificationToken

Field	Rules
expires_at	24h from creation for EMAIL_VERIFICATION; 1h for PASSWORD_RESET
is_used	Checked before honoring any verify/reset request; a used or expired token returns 400 TOKEN_INVALID_OR_EXPIRED regardless of which reason applies (does not reveal which, to avoid information leakage)

8.8 Field-Length & Format Quick Reference

Field type	Constraint
Names (first_name, last_name)	1-150 chars
email	RFC 5322 format, 254 char max
phone_number	$\wedge\backslash+?[1-9]\backslash d\{7,14\}\$$
Free text (bio, reason, notes)	1,000-2,000 char max, enforced both client- and server-side
Money (consultation_fee)	DecimalField(max_digits=8, decimal_places=2), non-negative

9. Security

9.1 JWT Security

- Signing algorithm: HS256 at MVP scale (single backend instance/cluster sharing one secret); revisit RS256 if the token ever needs to be verified by a separate service.
- `JWT_SIGNING_KEY` is a dedicated environment variable, distinct from Django's `SECRET_KEY` — rotating one never invalidates the other's dependents unexpectedly.
- Access tokens: 15 min. Refresh tokens: 7 days, rotated and blacklisted on use (Section 5.3–5.4).
- Tokens carry only `user_id` and `role` in the payload — no email, no PII, since JWT payloads are base64, not encrypted.

9.2 Permissions

Every view declares an explicit `permission_classes` — there is no “default open” endpoint by omission. `DEFAULT_PERMISSION_CLASSES` in settings is `IsAuthenticated`, so a view with no explicit override fails closed, not open, if a developer forgets to set it.

9.3 Password Security

- Hashing: Django default (`PBKDF2PasswordHasher`, upgradeable to `Argon2PasswordHasher` via `PASSWORD_HASHERS` with zero migration cost — Django rehashes transparently on next login).
- `AUTH_PASSWORD_VALIDATORS`: minimum length 12, common-password check, user-attribute-similarity check, numeric-only rejection.
- Current password required to set a new one (`POST /users/change-password/`); password reset (forgotten password) uses the token flow instead (5.6).

9.4 CORS

`django-cors-headers` with `CORS_ALLOWED_ORIGINS` set explicitly to the known React frontend origin(s) per environment — never `CORS_ALLOW_ALL_ORIGINS=True` outside local development. Credentials (`CORS_ALLOW_CREDENTIALS`) are not needed since auth is bearer-token-based, not cookie-based, which also sidesteps most CSRF concerns for the API itself.

9.5 CSRF

The JSON API is JWT-bearer-token authenticated and stateless, so Django's session-based CSRF protection is not applicable to `/api/v1/` endpoints and is explicitly exempted there. CSRF protection **remains fully enabled** for Django Admin (`/admin/`), which is session-cookie-authenticated — this is a deliberate split, not an oversight, and must not be “fixed” by globally disabling CSRF middleware.

9.6 Rate Limiting

Endpoint group	Limit
Login	5 / minute / IP
Registration	3 / hour / IP

Endpoint group	Limit
Password reset request	3 / hour / account, 10 / hour / IP
Resend verification	1 / minute / account
General authenticated API	60 / minute / user (DRF <code>UserRateThrottle</code>)
General anonymous API (public doctor browse)	30 / minute / IP (DRF <code>AnonRateThrottle</code>)

9.7 Input Validation

All input is validated through DRF serializers (Section 8) before touching the ORM — no view ever passes `request.data` directly into a model `.objects.create()`. `django-filter` validates filter/query parameters the same way, rejecting unrecognized filter fields rather than silently ignoring them.

9.8 SQL Injection Protection

The Django ORM parameterizes all queries by default; raw SQL is disallowed by the coding standard (3.7) except for a documented, reviewed performance exception, which — should one ever be added — must use parameterized `cursor.execute()` calls exclusively, never string-formatted SQL.

9.9 XSS Protection

The API returns `application/json`, never renders user input as HTML, so classic reflected/stored XSS in API responses is not applicable. Two specific surfaces still need attention: (1) Django Admin renders model fields as HTML by default — free-text fields (`bio`, `reason`, `doctor_notes`) are displayed as escaped text, never `mark_safe()`'d, in any custom admin display method; (2) the frontend (out of scope here) must escape/sanitize any rendered free-text field, which is called out explicitly in the API integration notes handed to the frontend team.

9.10 Secure File Uploads

- Profile pictures (`DoctorProfile.profile_picture`, `PatientProfile.profile_picture`): validated server-side by content-type sniffing (not just file extension), max 5 MB, restricted to `image/jpeg/image/png/image/webp`.
- Uploaded files are stored outside the web root's executable path (or in object storage — S3/GCS/Azure Blob — in production, per Section 14) and served through a randomized filename, never the client-supplied one, to prevent path traversal and overwrite attacks.
- Images are re-encoded via Pillow on upload (not merely copied), which both normalizes format and strips embedded metadata/EXIF and any non-image payload smuggled inside a polyglot file.

9.11 Environment Variables & Secrets Management

- `SECRET_KEY`, `JWT_SIGNING_KEY`, `DATABASE_URL`, `EMAIL_HOST_PASSWORD`, `REDIS_URL` — all environment-sourced, never defaulted to a non-empty value in `base.py` (a missing production env var should crash the app at boot, not silently run insecurely).
- Local development uses a git-ignored `.env` file; production uses the hosting platform's secret manager (Section 14).

- `.env.example` is the single documented list of required variables, kept in sync with `base.py` as a PR review checklist item (Section 11).

9.12 OWASP Top 10 — Applied Mapping

OWASP category	Mitigation in this design
Broken Access Control	Object-level permissions (5.9), explicit <code>permission_classes</code> everywhere (9.2), 404-not-403 on unauthorized object access
Cryptographic Failures	HTTPS-only in production, hashed passwords (9.3), no PII in JWT payload (9.1)
Injection	ORM parameterization (9.8), serializer validation (9.7)
Insecure Design	Service-layer business rules with explicit state machines (7.6), DB-level double-booking constraint (4.3.7) as defense-in-depth against a design bug, not just an implementation bug
Security Misconfiguration	12-factor config (3.10), <code>DEBUG=False</code> enforced in prod settings, explicit <code>ALLOWED_HOSTS</code>
Vulnerable Components	Dependency pinning (<code>requirements/*.txt</code>), <code>pip-audit/safety</code> in CI (Section 13)
Auth Failures	Rate limiting (9.6), token rotation/blacklisting (5.3–5.4), password policy (9.3)
Data Integrity Failures	Same DB constraint story as Insecure Design; signed JWTs
Logging & Monitoring Failures	Structured logging with request IDs (3.9), Sentry for exceptions
SSRF	No user-supplied URLs are ever fetched server-side in this system; flagged as a standing constraint if any future feature (e.g., webhook URLs) is added

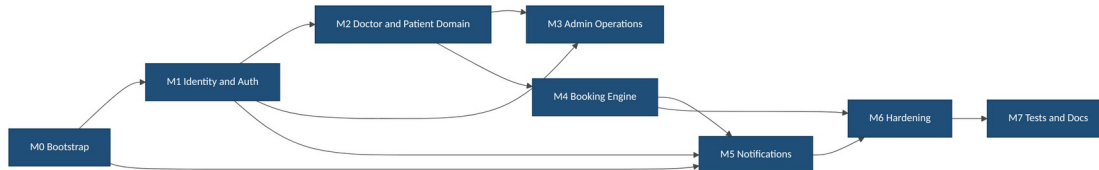
10. Development Roadmap

10.1 Milestones

Milestone	Contents	Complexity	Depends on
M0 — Project Bootstrap	Repo, Django project, settings split, Docker Compose (Postgres+Redis), CI skeleton, CORE app	Low	—
M1 — Identity & Auth	User, VerificationToken, registration, email verification, JWT login/refresh/logout, password reset	High	M0
M2 — Doctor & Patient Domain	Specialty, DoctorProfile, PatientProfile, DoctorAvailability, profile CRUD, doctor search	Medium	M1
M3 — Admin Operations	Approve/block endpoints, Django Admin configuration, AuditLog	Medium	M1, M2
M4 — Booking Engine	Slot computation, Appointment model + constraint, booking/cancel/reschedule/approve/reject/complete services	High	M2
M5 — Notifications	Notification model, Celery, email templates, wiring into M1/M3/M4 events	Medium	M0, M1, M4
M6 — Hardening	Rate limiting, throttling, security review pass (Section 9 checklist), load test the booking endpoint	Medium	M4, M5
M7 — Test Coverage & Docs	Close test gaps to coverage goal (Section	Medium	All above

Milestone	Contents	Complexity	Depends on
	13), finalize OpenAPI schema, developer guide (Section 15)		

10.2 Dependency Graph



10.3 Recommended Implementation Order

1. **M0** first and alone — nothing else can start without settings, Docker, and CI in place.
2. **M1** next, and treat it as blocking for the whole team — every other app depends on **User**.
3. Once M1's **User** model and JWT auth are merged, **M2 and M3 proceed in parallel** (different developers, Section 11) since **doctors/patients** and **admin-user-management** touch almost entirely disjoint files.
4. **M4** starts as soon as M2's **DoctorProfile/DoctorAvailability** are usable — it does not need to wait for M3.
5. **M5** can begin scaffolding (Celery config, base email templates) in parallel with M1–M4, but the actual trigger points wire in only once M1/M3/M4's events exist.
6. **M6 and M7 are continuous**, not a single end-of-project phase — the security checklist and test coverage goal apply incrementally to each milestone as it merges (Section 13's coverage gate runs in CI from M1 onward, not bolted on at the end).

10.4 Estimated Complexity Rationale

- **M1 and M4 are rated High** because they carry the system's two hardest correctness requirements: authentication/authorization (which everything else trusts) and the double-booking guarantee (which is the product's core promise). Budget the most senior engineering attention here, not the most calendar time necessarily — these need to be *right*, and reviewed accordingly.
- **M2, M3, M5, M6, M7 are Medium** — mostly well-trodden CRUD, configuration, and integration work with lower novel-risk surface, though not trivial.
- **M0 is Low** complexity but schedule-critical — a slow M0 delays everything after it, so it should not be treated as a "warm-up" task done casually.

11. Team Distribution

The backend team is 4 developers. Work is split by **Django app ownership**, matched to the dependency graph in Section 3.5 and the milestone graph in Section 10.2, so that two people are rarely editing the same file in the same week. **appointments** — the highest-dependency app — is deliberately split *within itself* between Dev 3 (patient-facing actions) and Dev 4 (doctor/admin-facing actions plus the shared booking-safety core), rather than given to one person, since it is the largest single milestone (M4).

11.1 Assignments

Developer 1 — Team Lead: Foundation & Identity

- **Responsibilities:** project bootstrap, settings/CI, the `core` app, all of `accounts` (User, auth, JWT, email verification, password reset, admin user-management endpoints, `AuditLog`).
- **Assigned modules:** `config/`, `core/`, `accounts/`.
- **Deliverables:** M0 in full; M1 in full; the `approve/block/unblock` endpoints of M3.
- **Estimated workload:** ~30% of total backend effort (M0 is foundational and M1 is the highest-risk milestone).
- **Dependencies:** blocks everyone else at the start (M1 must merge before M2–M5 can build on `User`); depends on no one.
- **Recommended branches:** `feature/project-bootstrap`, `feature/core-app`, `feature/accounts-user-model`, `feature/accounts-jwt-auth`, `feature/accounts-email-verification`, `feature/accounts-password-reset`, `feature/accounts-admin-actions`.
- **Merge order position:** **first** — every branch here merges to `develop` before Devs 2–4 branch off for anything touching `User`.

Developer 2 — Doctor Domain

- **Responsibilities:** `doctors` app end-to-end: `Specialty` CRUD, `DoctorProfile`, `DoctorAvailability`, doctor search/filter, slot computation (4.6).
- **Assigned modules:** `doctors/`.
- **Deliverables:** M2 (doctor half) in full; the slot-computation function consumed by Dev 4 in M4.
- **Estimated workload:** ~20%.
- **Dependencies:** needs Dev 1's `User`/JWT auth merged (M1) before starting; Dev 4 depends on Dev 2's slot computation and `DoctorAvailability` model being stable before M4's booking service can be finished.
- **Recommended branches:** `feature/doctors-specialty`, `feature/doctors-profile`, `feature/doctors-availability`, `feature/doctors-search`, `feature/doctors-slot-computation`.
- **Merge order position:** second, alongside Dev 3.

Developer 3 — Patient Domain & Patient-Side Booking

- **Responsibilities:** `patients` app (`PatientProfile`); within `appointments`, the patient-facing actions: create (book), list own, cancel, reschedule.

- **Assigned modules:** `patients/`, `appointments/` (booking, cancel, reschedule services + views).
- **Deliverables:** M2 (patient half); the patient-facing half of M4.
- **Estimated workload:** ~25%.
- **Dependencies:** needs M1 merged; needs Dev 2's `DoctorAvailability/slot` computation merged before booking service can be written; coordinates closely with Dev 4 on the shared `Appointment` model (see 11.3, merge order).
- **Recommended branches:** `feature/patients-profile`, `feature/appointments-model`, `feature/appointments-booking-service`, `feature/appointments-cancel-reschedule`.
- **Merge order position:** third — `appointments-model` branch must land before Dev 4's branches, since Dev 4 builds on the same model.

Developer 4 — Doctor/Admin-Side Appointments & Notifications

- **Responsibilities:** within `appointments`, the doctor/admin-facing actions (approve, reject, complete, admin all-appointments view); the entire `notifications` app (Celery setup, email templates, all notification-triggering wiring).
- **Assigned modules:** `appointments/` (approve/reject/complete services + views), `notifications/`.
- **Deliverables:** the doctor/admin half of M4; M5 in full.
- **Estimated workload:** ~25%.
- **Dependencies:** needs Dev 3's `appointments-model` branch merged first (shared model); needs Dev 1's Celery/environment scaffolding from M0.
- **Recommended branches:** `feature/appointments-doctor-actions`, `feature/appointments-admin-view`, `feature/notifications-celery-setup`, `feature/notifications-templates`, `feature/notifications-wiring`.
- **Merge order position:** fourth — after Dev 3's model branch, in parallel with the tail end of Dev 2's work.

11.2 Workload Summary

Developer	Focus	Est. share of effort
Dev 1 (Lead)	Foundation, identity, auth, admin actions	30%
Dev 2	Doctor domain, availability, search	20%
Dev 3	Patient domain, patient-side booking	25%
Dev 4	Doctor/admin-side appointments, notifications	25%

11.3 Merge Order (Critical Path)

1. Dev 1: `config + core` (M0) — everyone branches from this.
2. Dev 1: `accounts` User model + JWT auth (M1 core) — everyone who touches `request.user` branches from this next.
3. Dev 2 and Dev 3 proceed **in parallel** once step 2 merges: `doctors` app and `patients` app respectively touch disjoint files.
4. Dev 3: `appointments` **model** (the shared `Appointment` table + migration) merges before Dev 4 starts any `appointments`-related branch — this is the one deliberate serialization point in the whole plan, to avoid two developers writing conflicting migrations against the same new table.
5. Dev 3 (`booking/cancel/reschedule`) and Dev 4 (`approve/reject/complete/notifications`) proceed in parallel from step 4.
6. Dev 1: `admin` `approve/block` endpoints merge any time after step 2 (low conflict risk — touches `accounts` only).
7. Integration pass (all devs): `hardening` (M6) and `coverage/docs` (M7) once M4/M5 are both merged.

11.4 Code Review Checklist

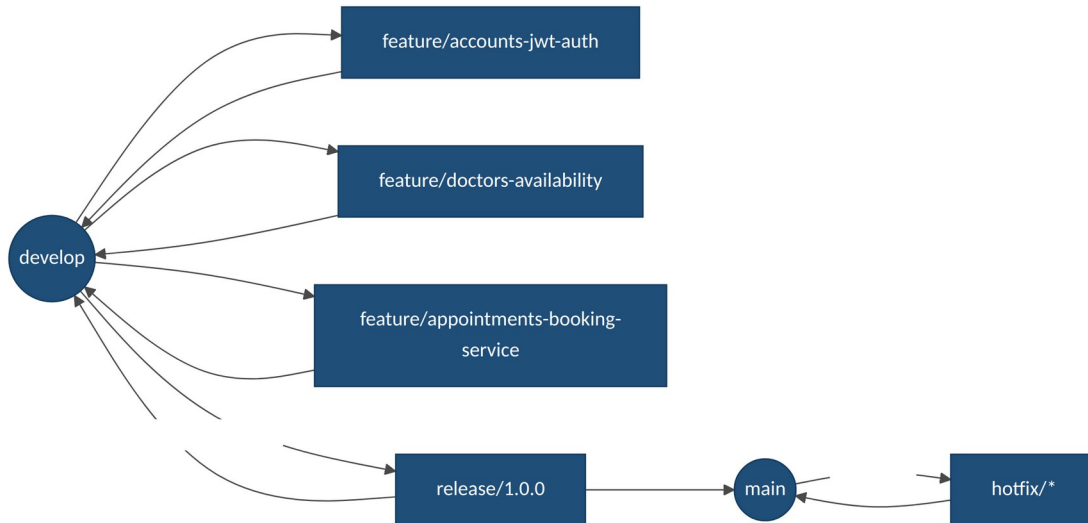
Every pull request, regardless of author, is checked against:

- ☐ No business logic in views or serializer `.save()` — lives in `services.py` (3.7).
- ☐ Every new list endpoint uses `select_related/prefetch_related` where FKs are serialized (4.5).
- ☐ Every new model change includes a migration, and the migration has been run against a fresh database, not just - - `faked` locally.
- ☐ New/changed endpoints are reflected in the OpenAPI schema (regenerate and diff, 6.7) and in this SDD if the contract changed.
- ☐ Permission classes explicitly declared on every new view (9.2) — no reliance on an implicit default beyond `IsAuthenticated`.
- ☐ New serializer fields have explicit validators, not just type hints (Section 8).
- ☐ Tests included: at minimum one happy-path and one permission-denied test per new endpoint (Section 13).
- ☐ No secrets, credentials, or `.env` values committed.
- ☐ `black/isort/flake8` (or `ruff`) pass in CI before review is requested.
- ☐ Any new/changed status transition on `Appointment` is reflected in the state-transition table (7.6), not just added as an ad-hoc branch.

12. Git Workflow

12.1 Branching Model

A simplified Git Flow is used: `main` is always production-deployable, `develop` is the integration branch, and all work happens on short-lived `feature/*` branches off `develop`.



12.2 Branch Naming

Type	Pattern	Example
Feature	feature/<app>-<short-description>	feature/appointments-booking-service
Bugfix (pre-release)	bugfix/<app>-<short-description>	bugfix/doctors-slot-off-by-one
Hotfix (production)	hotfix/<short-description>	hotfix/jwt-refresh-500
Release	release/<version>	release/1.0.0

12.3 Commit Conventions

Conventional Commits, enforced by a `commitlint` pre-commit hook:

`<type>(<scope>): <short summary>`

`<body – the "why", not a restatement of the diff>`

Type	Use
feat	New endpoint/capability
fix	Bug fix
refactor	No behavior change

Type	Use
<code>test</code>	Test-only change
<code>docs</code>	Documentation (including this SDD)
<code>chore</code>	Tooling, dependencies, CI

Example: `feat(appointments): add transactional double-booking guard`

12.4 Pull Request Workflow

1. Branch from up-to-date `develop`.
2. Keep the PR scoped to one milestone deliverable or smaller — a PR that touches more than one Django app is a signal to split it.
3. CI must pass (lint, tests, migration check, OpenAPI schema diff) before requesting review.
4. Minimum **one** approval from another developer; PRs touching `accounts` or the `Appointment` model require the Team Lead's (Dev 1's) approval specifically, given their blast radius.
5. Squash-merge to `develop` with a Conventional Commits summary as the squash message.

12.5 Code Review Process

Review against the checklist in Section 11.4. Reviewers respond within one business day; author addresses feedback via new commits (not force-push) until approval, then squash-merges. Disagreements that aren't resolved in two review rounds escalate to a short synchronous discussion with the Team Lead rather than an unbounded comment thread.

12.6 Release Strategy

- Cut a `release/<version>` branch from `develop` when a milestone group is feature-complete and passing the M6 hardening checklist.
- Only bug fixes land on the release branch; new features continue on `develop` for the next release.
- On release sign-off: merge `release/<version>` into both `main` (tag `v<version>`) and back into `develop` (so any release-branch fixes aren't lost).
- `main` merges trigger the production deploy pipeline (Section 14); `develop` merges trigger deploy to the staging environment.

13. Testing Strategy

13.1 Testing Pyramid for This Project

Layer	What it covers	Tooling
Unit tests	<code>services.py</code> functions in isolation (validation logic, state transitions, slot computation)	<code>pytest</code> , <code>pytest-django</code>
Integration tests	Service functions against a real (test) database, including transaction/constraint behavior	<code>pytest-django</code> , Django's <code>TransactionTestCase</code> where <code>select_for_update/constraint</code> behavior must be exercised
API tests	Full request/response cycle through DRF views, including serialization and permission enforcement	<code>pytest-django</code> + DRF <code>APIClient</code>
Permission/authentication tests	Every role × every protected endpoint	<code>pytest-django</code> + DRF <code>APIClient</code> , parametrized

13.2 Unit Tests

Focus areas: `appointments.services` state-transition table (every legal and illegal transition), `doctors.services.compute_available_slots()` (boundary conditions: slot exactly at availability edge, fully booked day, no availability defined), `accounts.services` token expiry logic. Unit tests must not hit the database where avoidable — pure logic functions are tested with plain Python objects/mocks.

13.3 Integration Tests

The one test suite that matters most in this entire document: **concurrent booking**. Using `TransactionTestCase` and multiple threads/`ThreadPoolExecutor` (or `pytest-django`'s `django_db_blocker` with real transactions), fire N simultaneous booking requests at the same doctor/date/slot and assert exactly one succeeds and the rest receive `409 SLOT_UNAVAILABLE`. This test must run in CI on every PR that touches `appointments/` — it is the automated proof of Section 7.5's guarantee, not just a manual QA check.

13.4 API Tests

Every endpoint in Section 6.8 gets at minimum: - One authenticated happy-path test asserting status code and response shape. - One validation-error test (bad input → `400` with expected `details` keys). - One not-found test (`404` for a nonexistent or invisible-to-this-user object ID).

13.5 Permission Tests

A parametrized matrix test iterates every (role, endpoint) pair from Section 5.8's access table and asserts the expected status code — this single parametrized test file is the automated enforcement of that table, and must be updated in the same PR any time the access table changes.

13.6 Authentication Tests

- Registration → verification token created → login blocked until verified → login succeeds after verification.
- Login rejects wrong password, unverified email, blocked account (each with the specific error code from Section 6.4).
- Refresh token rotation: reusing an already-rotated refresh token fails.
- Password reset: request always returns 200 regardless of email existence; confirm rejects expired/used tokens.

13.7 Appointment Tests

- Full lifecycle happy path: book → approve → complete.
- Full rejection path: book → reject (with reason) → slot freed → rebookable.
- Cancellation frees the slot; rescheduling preserves `previous_appointment` linkage and leaves the original untouched on failure (7.8).
- Every illegal state transition from 7.6's table returns 409 `INVALID_STATE_TRANSITION`.

13.8 Factories & Fixtures

`factory_boy` factories for every model, defined in each app's `tests/factories.py`:

```
class DoctorProfileFactory(factory.django.DjangoModelFactory):
    class Meta:
        model = DoctorProfile
        user = factory.SubFactory(UserFactory, role="DOCTOR",
account_status="ACTIVE")
        specialty = factory.SubFactory(SpecialtyFactory)
        license_number = factory.Sequence(lambda n: f"LIC-{n:06d}")
```

Shared fixtures (`conftest.py` at the project root): `api_client`, `authenticated_patient_client`, `authenticated_doctor_client`, `authenticated_admin_client` — every API test starts from one of these rather than re-authenticating inline.

13.9 Coverage Goals

Area	Target
<code>services.py</code> across all apps	≥ 90%
Overall project	≥ 85%
<code>appointments</code> app specifically	≥ 95% (highest-risk code in the system)

Coverage is measured with `coverage.py/pytest-cov` and enforced in CI (`--cov-fail-under`); a PR that drops overall coverage below the gate fails CI regardless of whether the new code itself is tested, forcing the author to either test it or explicitly justify an exclusion in review.

13.10 Testing Tools Summary

Tool	Purpose
<code>pytest + pytest-django</code>	Test runner
<code>DRF APIClient</code>	HTTP-level API tests
<code>factory_boy + Faker</code>	Test data generation
<code>coverage.py / pytest-cov</code>	Coverage measurement and CI gate
<code>freezegun</code>	Freezing time for date/slot boundary tests
<code>pytest-xdist</code>	Parallel test execution to keep CI fast as the suite grows

14. Deployment & DevOps Strategy

(Note: the original brief's section numbering skipped from 13 to 15. This section is added as 14 — Deployment & DevOps — to fill that gap, since a backend SDD handed to developers before implementation begins is materially incomplete without stating how the thing actually ships. Renumber if the omission was intentional.)

14.1 Environments

Environment	Purpose	Deploy trigger
Local	Developer machines, Docker Compose	Manual
Staging	Integration testing, frontend team's target for API integration	Merge to develop
Production	Live system	Merge to main (tagged release)

14.2 Containerization

- The Django app is packaged as a Docker image built from a multi-stage `Dockerfile` (build stage installs dependencies; final stage is a slim runtime image running `gunicorn`).
- `docker-compose.yml` (local/dev only) runs: `web` (Django), `db` (PostgreSQL 16), `redis`, `worker` (Celery), `beat` (Celery beat, if scheduled tasks like appointment reminders are enabled — Section 16).
- Production uses the same image built in CI, deployed to whatever the team's hosting target is (a managed container platform — the specific provider is an infrastructure decision outside this document's scope, but the image and environment-variable contract are the same regardless of provider).

14.3 CI/CD Pipeline



14.4 Database Migrations in Production

- Migrations run as a distinct release step **before** the new application version receives traffic, never as an app-startup side effect (which risks two instances racing to migrate during a rolling deploy).
- Backward-incompatible migrations (column removal, type change) are split into two deploys (add-and-dual-write, then remove) rather than one — standard practice, called out explicitly since the `Appointment` table's constraint (4.3.7) is exactly the kind of migration that must never be applied carelessly against a live table with existing rows.

14.5 Configuration & Secrets in Production

- All variables from Section 9.11 are injected via the hosting platform's secret manager at deploy time, never baked into the image.

- `DEBUG=False`, explicit `ALLOWED_HOSTS`, `SECURE_SSL_REDIRECT=True`, `SESSION_COOKIE_SECURE/CSRF_COOKIE_SECURE=True` (relevant to Django Admin's session auth, Section 9.5) are enforced in `config/settings/prod.py` and covered by a settings-sanity check that runs as part of the deploy pipeline (`manage.py check --deploy`).

14.6 Monitoring & Observability

- Application errors: Sentry (or equivalent), wired via `sentry-sdk`'s Django integration.
- Structured logs (3.9) shipped to the platform's log aggregator.
- Health check endpoint (`GET /api/v1/health/`, unauthenticated) verifies DB and Redis connectivity for load-balancer/orchestrator liveness checks.
- Basic uptime/latency monitoring on the booking endpoint specifically, given its centrality to the product's core promise.

14.7 Backups & Disaster Recovery

- Automated daily PostgreSQL backups with point-in-time recovery enabled, retained per the hosting platform's standard window (minimum 7 days for MVP).
- Backup restore is tested at least once before go-live — an untested backup is not a backup.

15. Developer Guide

15.1 Project Setup

```
git clone <repo-url> medbook_backend && cd medbook_backend
cp .env.example .env # fill in local values
docker compose up -d db redis # start dependencies only
python -m venv .venv && source .venv/bin/activate
pip install -r requirements/dev.txt
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
# in a second terminal, for anything touching notifications:
celery -A config worker -l info
```

15.2 Folder Explanation

See Section 3.4 for the full tree. In short: `config/` is Django's project package (settings, root URLs, Celery/ASGI/WSGI entry points); every business domain is its own top-level app (`accounts`, `doctors`, `patients`, `appointments`, `notifications`); `core/` holds cross-cutting code with no business meaning of its own (base model, pagination, permissions, exception handling).

15.3 Environment Variables

Variable	Example (local)	Notes
DEBUG	True	Never True in staging/prod
SECRET_KEY	(random 50-char string)	Django's cryptographic signing key
JWT_SIGNING_KEY	(separate random string)	Do not reuse SECRET_KEY
DATABASE_URL	postgres:// medbook:medbook@local host:5432/medbook	
REDIS_URL	redis:// localhost:6379/0	Celery broker + cache
EMAIL_HOST, EMAIL_PORT, EMAIL_HOST_USER, EMAIL_HOST_PASSWORD	(provider-specific)	Local dev can use <code>django.core.mail.backends.console.EmailBackend</code> instead
CORS_ALLOWED_ORIGINS	http://localhost:5173	React dev server origin
ALLOWED_HOSTS	localhost,127.0.0.1	Comma-separated
MAX_BOOKING_HORIZON_DAYS	90	Business-rule config, not a secret (2.9, 2.8)

15.4 Coding Standards & Naming Conventions

See Section 3.7 and 3.8 in full — this guide does not repeat them, it points here so there is exactly one place they can drift out of sync.

15.5 Developer Onboarding Checklist (First Day)

1. Read this document in full (yes, all of it, before writing code).
2. Complete 15.1 (Project Setup) and confirm `python manage.py test` passes locally.
3. Read Section 11 to find your app ownership and Section 12 for the branch you'll be working from.
4. Read Section 5 and Section 7 closely if your work touches `accounts` or `appointments` — these carry the highest correctness bar in the system.
5. Open a small first PR (e.g., a test or a minor endpoint) to confirm your local CI/lint/pre-commit setup works before taking on a milestone deliverable.

15.6 README Structure (Repository Root)

Medical Appointment Booking System — Backend

1. Overview (one paragraph, link to this SDD)
2. Requirements (Python/Postgres/Redis versions)
3. Local setup (mirrors 15.1)
4. Running tests
5. Running Celery
6. Environment variables (link to `.env.example`)
7. API documentation (link to OpenAPI schema / drf-spectacular UI)
8. Contributing (link to Section 12 — Git Workflow)
9. License

15.7 Best Practices Recap

- Fat models, thin views, explicit services (3.7). If you're not sure where logic goes, it goes in `services.py`.
- Every new endpoint needs: a serializer, an explicit permission class, at least two tests (13.4), and an OpenAPI schema entry that isn't hand-written but generated from the view itself.
- Never trust client-supplied `role` or `account_status` in a request body — these are set exclusively by server-side logic (5.1, 7.2).
- When in doubt about a business rule not covered here, check Section 2.9 (open questions) before inventing a new one — if it isn't there either, ask the Team Lead and add it to this document.

16. Future Improvements

Ordered by priority — each entry states the benefit and the complexity of adding it on top of the architecture already specified, so nothing here requires a rewrite, only extension.

Priority	Improvement	Benefit	Complexity
High	SMS/push notification channel alongside email	Reduces missed-appointment no-shows; <code>Notification.channel</code> field already anticipates this (Section 4.3.8 can add a <code>channel</code> column)	Medium — new provider integration, no schema redesign
High	Appointment reminders (Celery beat scheduled task, e.g., 24h before)	Directly reduces no-shows, the single biggest driver of wasted doctor capacity in any booking system	Low — <code>Notification</code> model and Celery infrastructure already exist (Section 14.2 mentions <code>beat</code>)
High	NO_SHOW status distinct from CANCELLED	Lets doctors and admins measure a real no-show rate instead of it hiding inside “cancelled”	Low — one more row in the state-transition table (7.6)
Medium	Doctor ratings & reviews	Patient trust signal, differentiates doctor search beyond specialty/name	Medium — new model + moderation workflow, hangs cleanly off <code>Appointment</code> (only reviewable after COMPLETED)
Medium	Payments & billing (consultation fee collection)	Turns <code>consultation_fee</code> from informational into transactional	High — PCI scope, refund/cancellation-fee policy, a new payment provider integration and its own failure-mode design
Medium	Multi-specialty doctors	Matches real-world practice more closely	Medium — <code>DoctorProfile.specialty</code> FK becomes a many-to-many; flagged already in Section 4.7
Medium	Materialized generated-slots table	Performance at high scale (Section 4.7)	Medium — write-path complexity in exchange

Priority	Improvement	Benefit	Complexity
			for read-path speed; only worth doing once profiling justifies it
Low	Multi-timezone support	Needed if the platform expands beyond a single-timezone deployment (open question 2.9.1)	Medium-High — touches every date/time field's semantics; better to design in deliberately than retrofit
Low	Telemedicine / video consultation	New product surface, not just a backend extension	High — real-time infrastructure, an entirely different NFR profile
Low	Multi-clinic / multi-tenant support	Only relevant if the product expands to serve multiple independent clinics on one deployment	High — touches authorization model throughout; would likely warrant its own SDD revision
Low	In-app real-time chat (patient ↔ doctor)	Explicitly out of scope per product requirements	High — new domain, new NFRs (message delivery guarantees), best treated as a separate service

How to use this table: nothing here should be built opportunistically mid-sprint just because a developer has spare time — each row, if pursued, gets its own short design note reviewed against this document's architecture (Section 3) and data model (Section 4) before implementation starts, exactly as this document was produced before Section 1's MVP began.